LEXIARBITER

環境設置

- **什麼是「環境」（Environment）？**

  在程式世界裡，要讓一段程式碼順利執行，電腦中也需要安裝特定的工具（例如 Python 或 Java 軟體）、翻譯機，以及其他前人寫好的工具包（套件）。這些為了讓程式能順利運作而準備好的軟體與設定總和，就稱為「環境」。

- **什麼是「虛擬環境」（Virtual Environment）？**

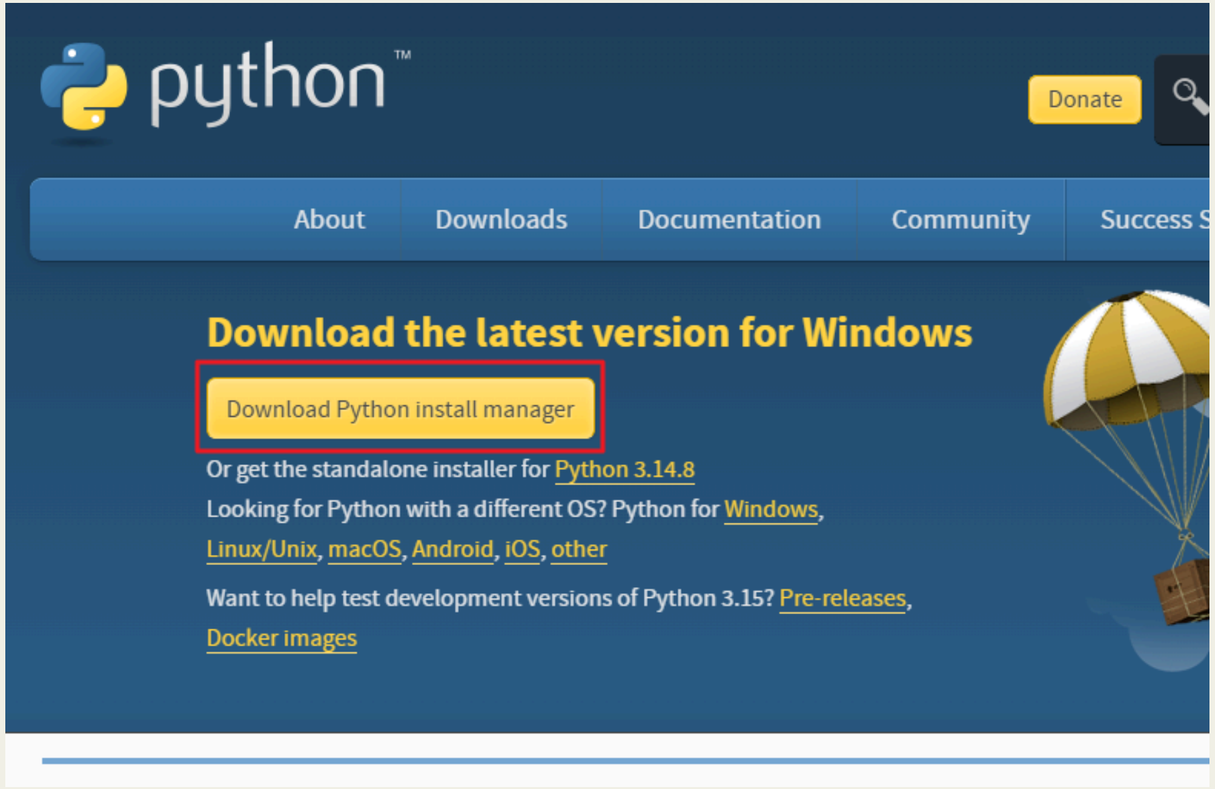
  虛擬環境是在電腦系統中，特地建立出的一個獨立且隔離的資料夾空間。這個空間擁有自己專屬的程式語言執行檔與套件庫。任何在這個虛擬環境內部安裝、更新或刪除的軟體套件，都被限制在這個空間內，完全不會影響到電腦原本的系統，也不會干擾電腦上的其他資料夾與專案。

- **為什麼要安裝「虛擬環境」？**

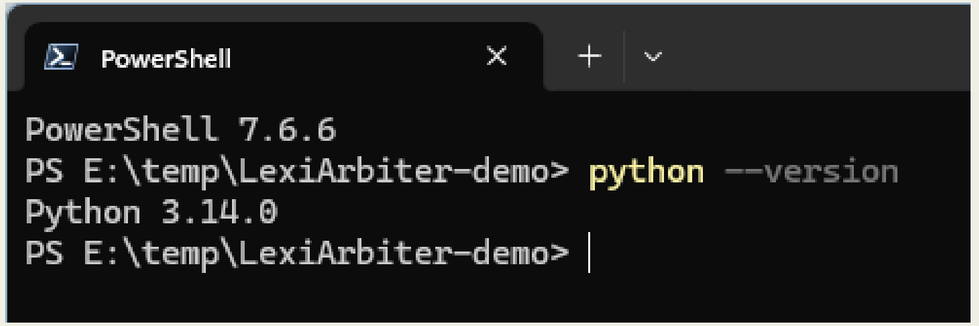
  安裝虛擬環境是為了替專案建立一個「獨立隔離的執行空間」。防止不同專案的軟體版本互相干擾（避免版本衝突），並能精確統一軟體清單，確保團隊所有成員都在完全一致的條件下執行程式，解決「別人電腦能跑，我的卻會報錯」的協作問題。

STEP1-PYTHON安裝

安裝 python，**至少安裝 python 3.10以上**，下載連結：python下載

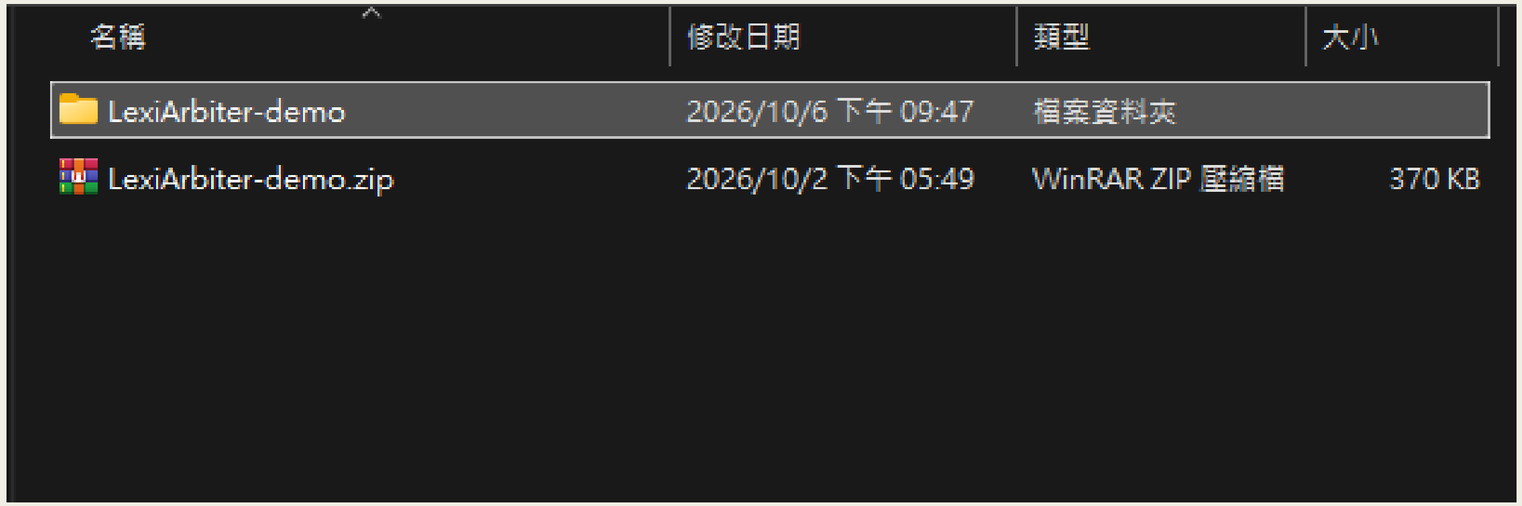


不確定版本可以用：*python --version* 來確認
(也可以用來確認 python 的安裝有沒有成功)



STEP2-解壓縮程式碼

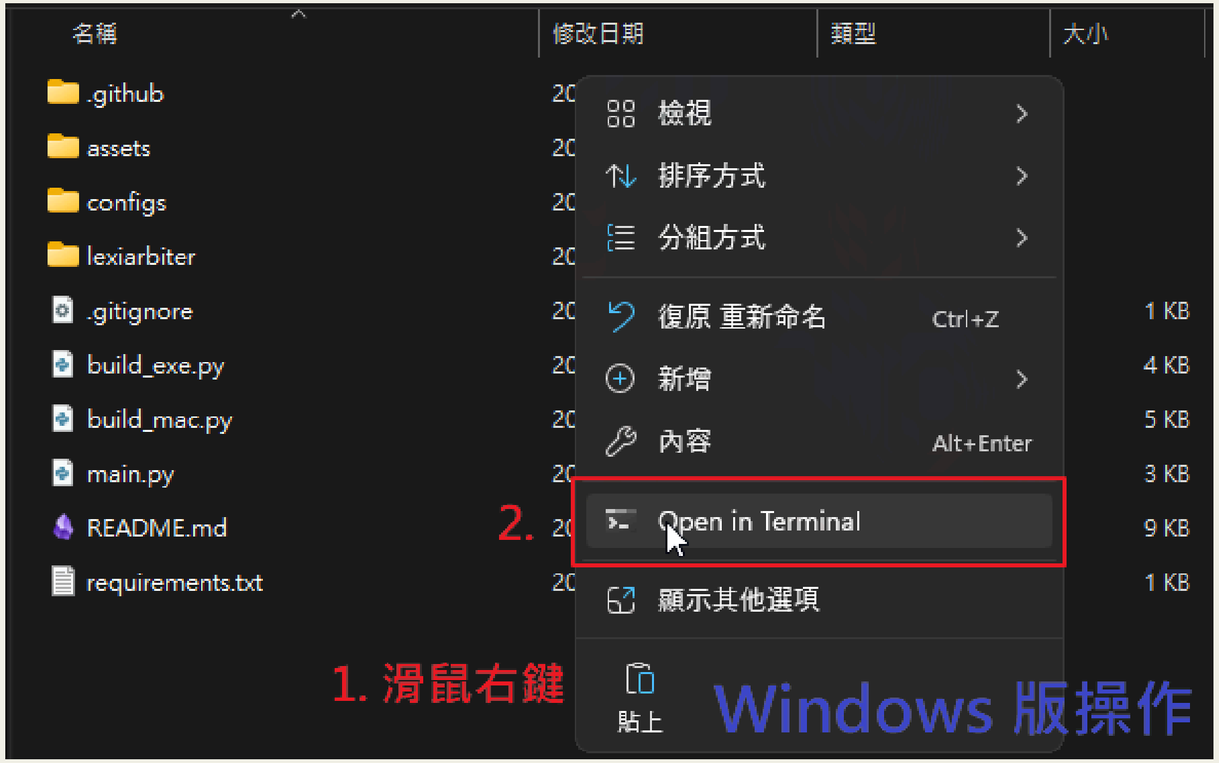
把拿到的專案程式碼壓縮檔解壓縮到你想要的位置 (桌面 or D槽“D:/” 等位置)



STEP3-打開終端機

找到剛剛解壓縮出來的專案資料夾，用終端機 (terminal) 打開：

- Windows可以直接在專案資料夾按滑鼠右鍵後選“在此開啟終端機”，或是用指令：“***cd [專案資料夾路徑]***”
- mac可在 Finder 壓住專案資料夾不放，往下拖曳到 Dock 上的終端機圖示上再放開。 這會立刻彈出一個全新的終端機視窗，而且路徑已經自動綁定在該資料夾下。或是打開終端機後在終端機輸入“***cd*** ”(注意 ***cd*** 後面一定要包含一個空格)，從 Finder 把專案資料夾直接拖進終端機視窗裡放開，會自動帶入資料夾路徑，再按 enter 就可以進到專案資料夾中。

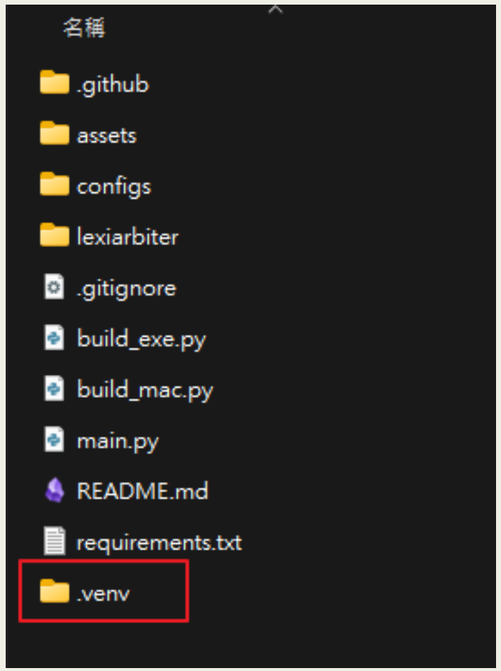




STEP4-建立虛擬環境

確認終端機 (terminal) 進到專案資料夾中後，就可以為這個專案建立專屬的虛擬環境了。
在終端機輸入指令：*python3.14 -m venv .venv* (如果你的 python 是其他版本在 *python* 後面帶你所安裝的版本，或是不輸入版本號用系統預設版本來建立虛擬環境)

完成後專案資料夾下就會新增一個.venv的資料夾



mac可用終端機輸入指令：*ls -la* 確認
(.venv Finder 預設會隱藏)

```
hanatai@HanaTaideMacBook-Air LexiArbiter-main % ls -la
total 72
drwxr-xr-x@ 13 hanatai  staff   416 10  5 14:34 .
drwx------+ 85 hanatai  staff  2720 10  5 18:56 ..
drwxr-xr-x@  3 hanatai  staff    96 10  1 05:56 .github
-rw-r--r--@  1 hanatai  staff   217 10  1 05:56 .gitignore
drwxr-xr-x   6 hanatai  staff   192 10  5 14:18 .venv
drwxr-xr-x@  4 hanatai  staff   128 10  1 05:56 assets
-rw-r--r--@  1 hanatai  staff  3690 10  1 05:56 build_exe.py
-rw-r--r--@  1 hanatai  staff  4731 10  1 05:56 build_mac.py
drwxr-xr-x@  4 hanatai  staff   128 10  1 05:56 configs
drwxr-xr-x@  6 hanatai  staff   192 10  1 05:56 lexiarbiter
-rw-r--r--@  1 hanatai  staff  2113 10  1 05:56 main.py
-rw-r--r--@  1 hanatai  staff  9134 10  1 05:56 README.md
-rw-r--r--@  1 hanatai  staff    36 10  1 05:56 requirements.txt
hanatai@HanaTaideMacBook-Air LexiArbiter-main % source .venv/bin/activate
(.venv) hanatai@HanaTaideMacBook-Air LexiArbiter-main %
```

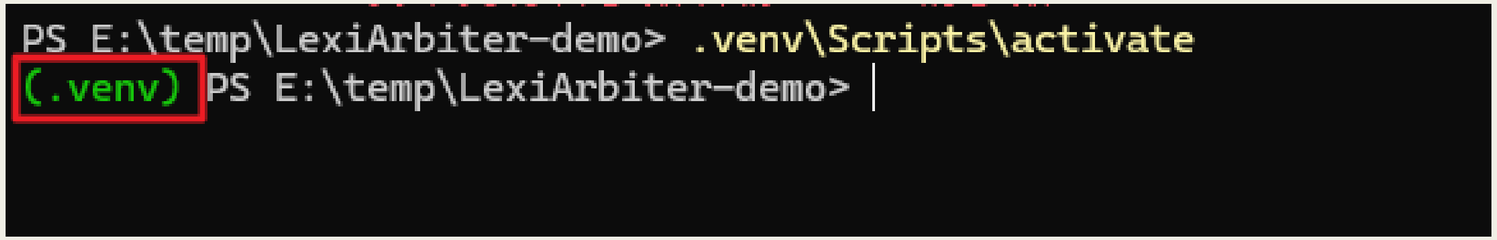
```
PowerShell
PowerShell 7.6.6
PS E:\temp\LexiArbiter-demo> python --version
Python 3.14.0
PS E:\temp\LexiArbiter-demo> python3.14 -m venv .venv
PS E:\temp\LexiArbiter-demo>
```

STEP5-啟動虛擬環境

建立好虛擬環境後就能啟動它來安裝這專案所需的套件了，另外之後要執行專案程式時也要記得先進入虛擬環境後才能正常執行專案。
要啟動虛擬環境，於終端機輸入並執行指令：

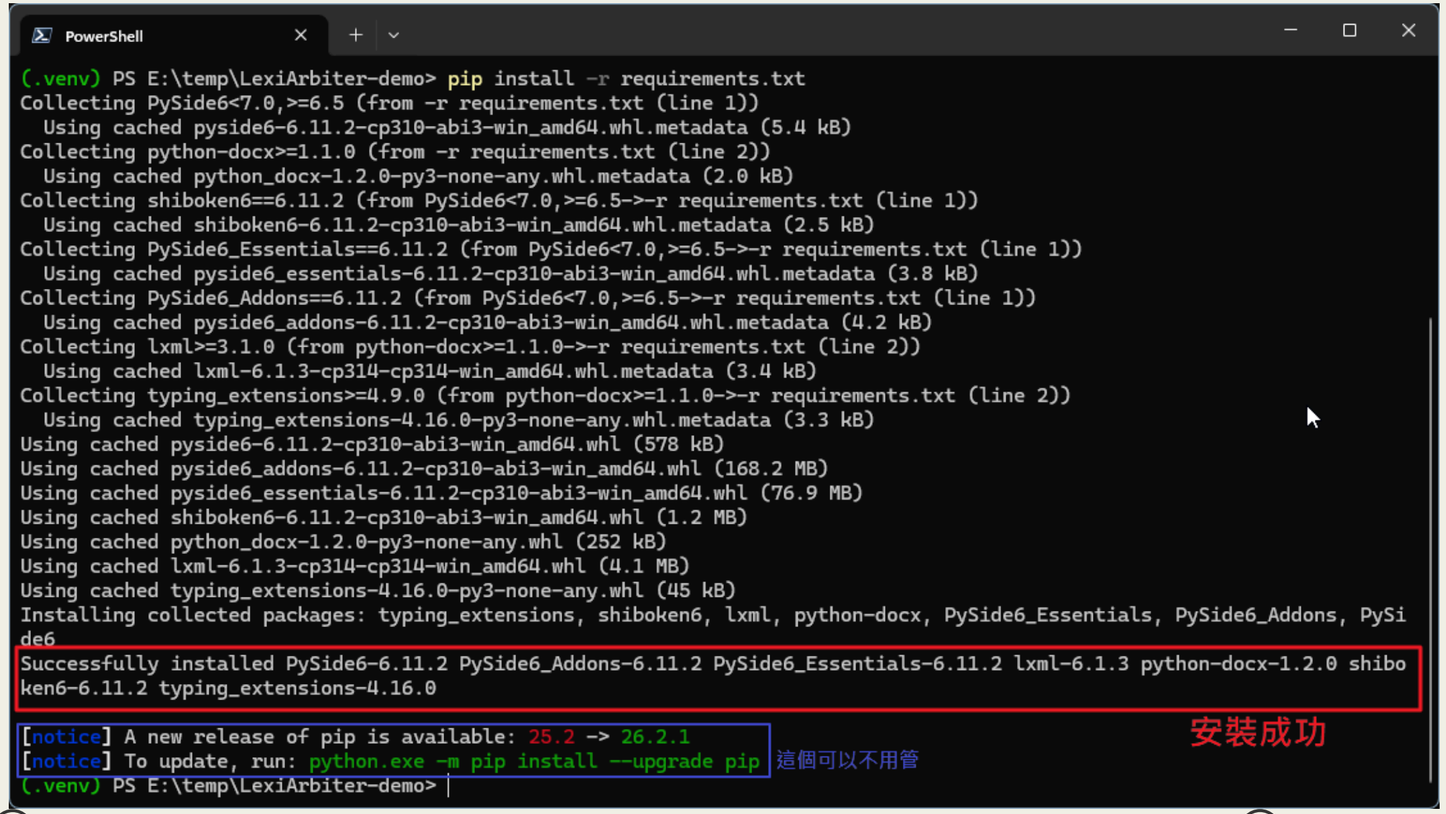
- Windows：*.venv\Scripts\activate*
- MAC：*source .venv/bin/activate*

啟動後 command line 前面會出現 *(.venv)* 表示虛擬環境成功啟動



STEP6.1-安裝套件

啟動虛擬環境後就可以安裝套件了，範例程式中已準備了這個專案所會用到的套件清單(requirements.txt)，只要下指令依照清單來安裝就能建置好程式的執行環境。
於終端機中輸入並執行：*pip install -r requirements.txt* 後等待安裝完成。



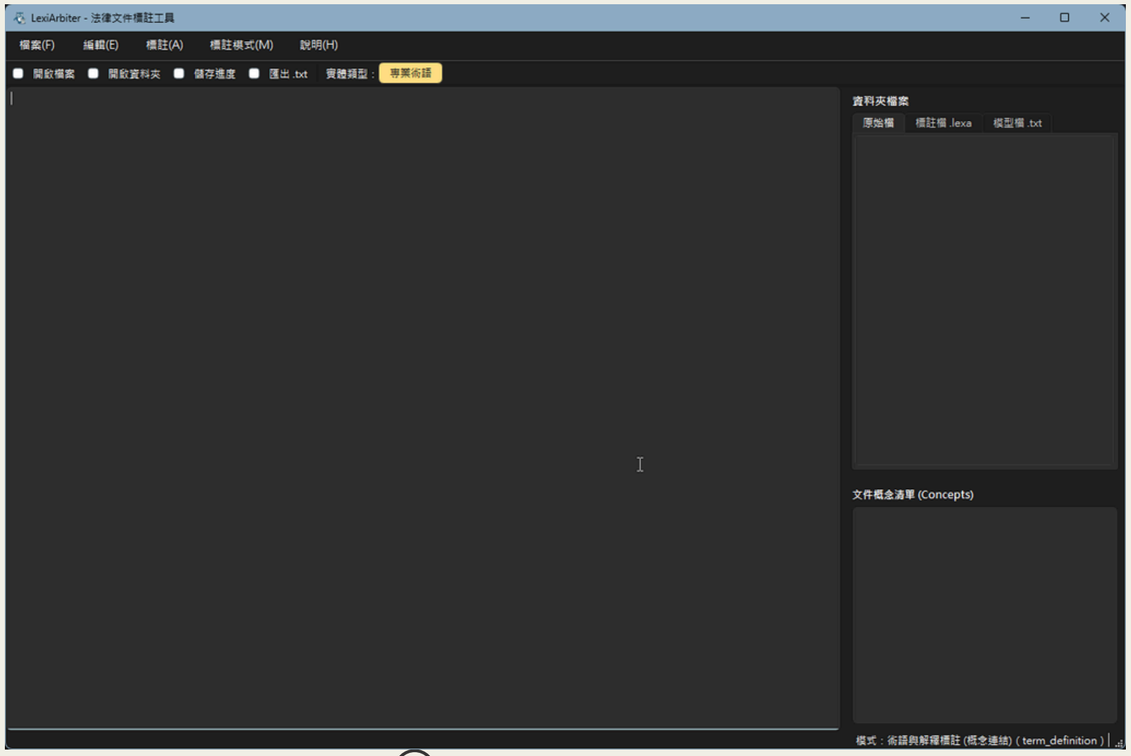
STEP6.2-安裝套件(離線)

啟動虛擬環境後就可以安裝套件了，範例程式中已準備了這個專案所會用到的套件清單(requirements.txt)，只要下指令依照清單來安裝就能建置好程式的執行環境。
於終端機中輸入並執行：
pip install --no-index --find-links=./offline_packages -r requirements.txt 後等待安裝完成。

STEP7-啟動程式

套件安裝完成後就可以啟動程式了。
在終端機中輸入：*python main.py* 就會出現程式視窗

```
(.venv) PS E:\temp\LexiArbiter-demo> python main.py
```

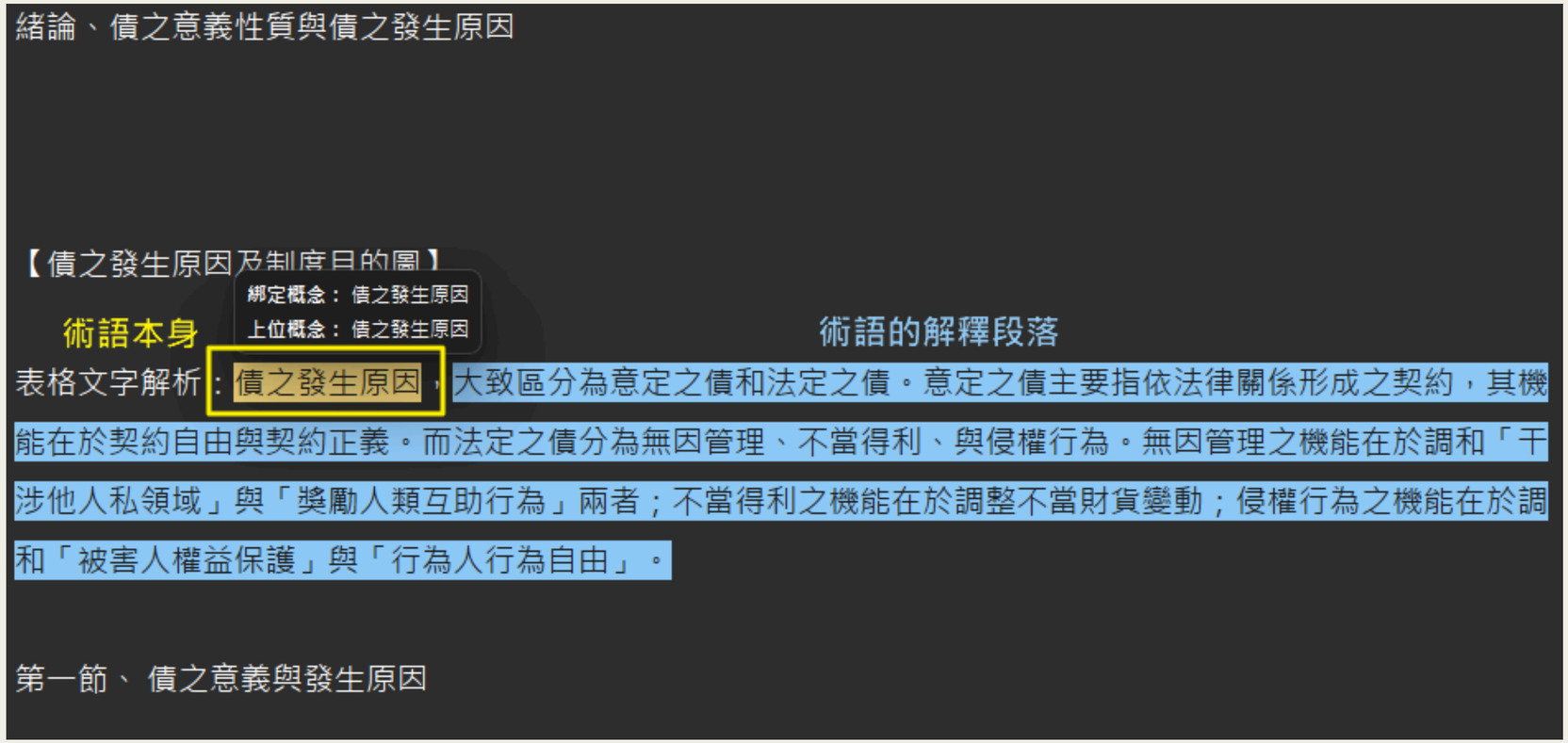


之後要啟動程式時要記得：

1. **開啟終端機進入專案資料夾下 (Step 3)**
2. **啟動虛擬環境 (Step 5)**
3. **執行程式 (Step 7)**

標註任務

在文件中標出一個術語以及這個術語對應的解釋段落，並且記錄術語本身所對應的概念以及上位概念。



標註任務

在文件中標出一個術語以及這個術語對應的解釋段落，並且記錄術語本身所對應的概念以及上位概念。

術語本身

- 術語的概念
- 術語的上層概念

術語對應的解釋段落

一個標註

GUI 101

GUI 程式的基本邏輯

0.1 一般程式 VS GUI 程式

平常寫的 Python 程式是從第一行跑到最後一行，跑完就結束。
GUI 程式不一樣：它先把畫面「蓋好」，然後進入一個叫 event loop 的迴圈，一直等，直到你關閉視窗為止。等待的過程中，只要使用者做了某個動作（按按鈕、拖選文字、按右鍵），程式才會去執行對應的函式，執行完又回去繼續等。
可以想像成餐廳的服務生：開店時先把桌椅擺好（蓋畫面），然後就站在旁邊等；客人舉手（事件），服務生才過去處理，處理完再回去等下一位。

main.py 啟動 ➡ MainWindow.init() 蓋畫面：建立按鈕、選單、編輯器，並「登記」每個事件發生時要呼叫哪個函式 ➡ app.exec() 進入 event loop，開始等



所以在 GUI 程式裡，函式大多是**「事先登記好，等事件發生時由 Qt 幫你呼叫」**。

0.2 三個角色：元件、SIGNAL、處理函式

- 元件（widget）：就是畫面上看得到的東西，如工具列按鈕、編輯器 self.editor、右側檔案清單 self.file_panel等。
- 事件（signal）：元件「發生某件事」時發出的通知，如 triggered（按下工具列 / 選單項目）、clicked（按下 QPushButton）、selection_changed（選取範圍改變）。
- 處理函式（slot）：事件發生時要執行的函式，如 MainWindow.action_open。

把這三者串起來的就是 connect()：

元件.signal.connect(處理函式)

意思是「登記：當這個元件發生這件事時，請呼叫這個函式」。
登記的當下，函式不會執行。

0.3 FOR EXAMPLE...

在 template 程式碼的 MainWindow._build_toolbar (app.py)裡，「開啟檔案」按鈕只有兩行：

```
a_open = tb.addAction(_swatch_icon("#FFFFFF"), "開啟檔案")
a_open.triggered.connect(self.action_open)
```

① 建立元件：在工具列 tb 上加一個按鈕

② 登記：按下（triggered）時呼叫 action_open（處理函式）

③ 處理函式 MainWindow.action_open 定義在 MainWindow 的其他地方。程式啟動時它不會跑；之後每按一次按鈕，Qt 就呼叫它一次。

程式啟動時（只跑一次）：
MainWindow.init → _build_toolbar

- ① 建立「開啟檔案」按鈕
- ② a_open.triggered.connect(self.action_open)

之後每次使用者按下按鈕：
按按鈕 ⇢ emit triggered ⇢ 接到 MainWindow.action_open()
③ 跳出選擇檔案的視窗

0.4 EVENTS IN TEMPLATE...

事件不一定是「按按鈕」。下面是 MainWindow.___init___ 裡實際的其中幾條接線：

- 使用者按下工具列按鈕 → 按鈕發出 triggered signal → 接到 action_open 處理。
- 使用者文字選取範圍改變 → self.editor 發出 selection_changed(s, e) signal → 接到 _on_selection_changed 處理。
- 每 60 秒自動存檔 (不需要使用者做任何事) → self._autosave_timer (計時器) 發出 timeout signal → 接到 _autosave_tick 處理。

所以核心概念其實就是：**「哪個事件發生了 → 它被 connect 到哪個函式 → 那個函式做了什麼」**。
※ 想加新功能、改流程、改接線就是去找你想改的是什麼 event 以及 connect 到了哪個函式。

TIPS

一些幫助你探索、開發更順手的方式

TIPS

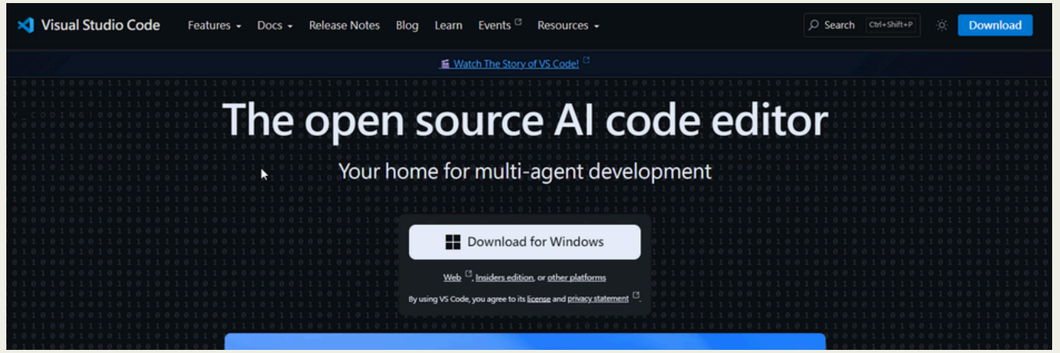
接下來的介紹只是經驗分享，提供給一些沒有經驗的人一些參考，但並不一定要這麼做，你可以尋找你喜歡的方式。

工欲善其事必先利其器，寫程式時如果有個好用的工具幫我們把會用到的功能例如：

- 編輯器：打程式碼的地方，會用顏色標示不同語法，方便閱讀。
- 執行：一鍵執行寫好的程式，看結果。
- Debug：程式出錯時，幫你找出是哪一行出問題。
- 自動提示：打字時自動建議接下來可能要寫的內容，也會提醒拼錯的地方。

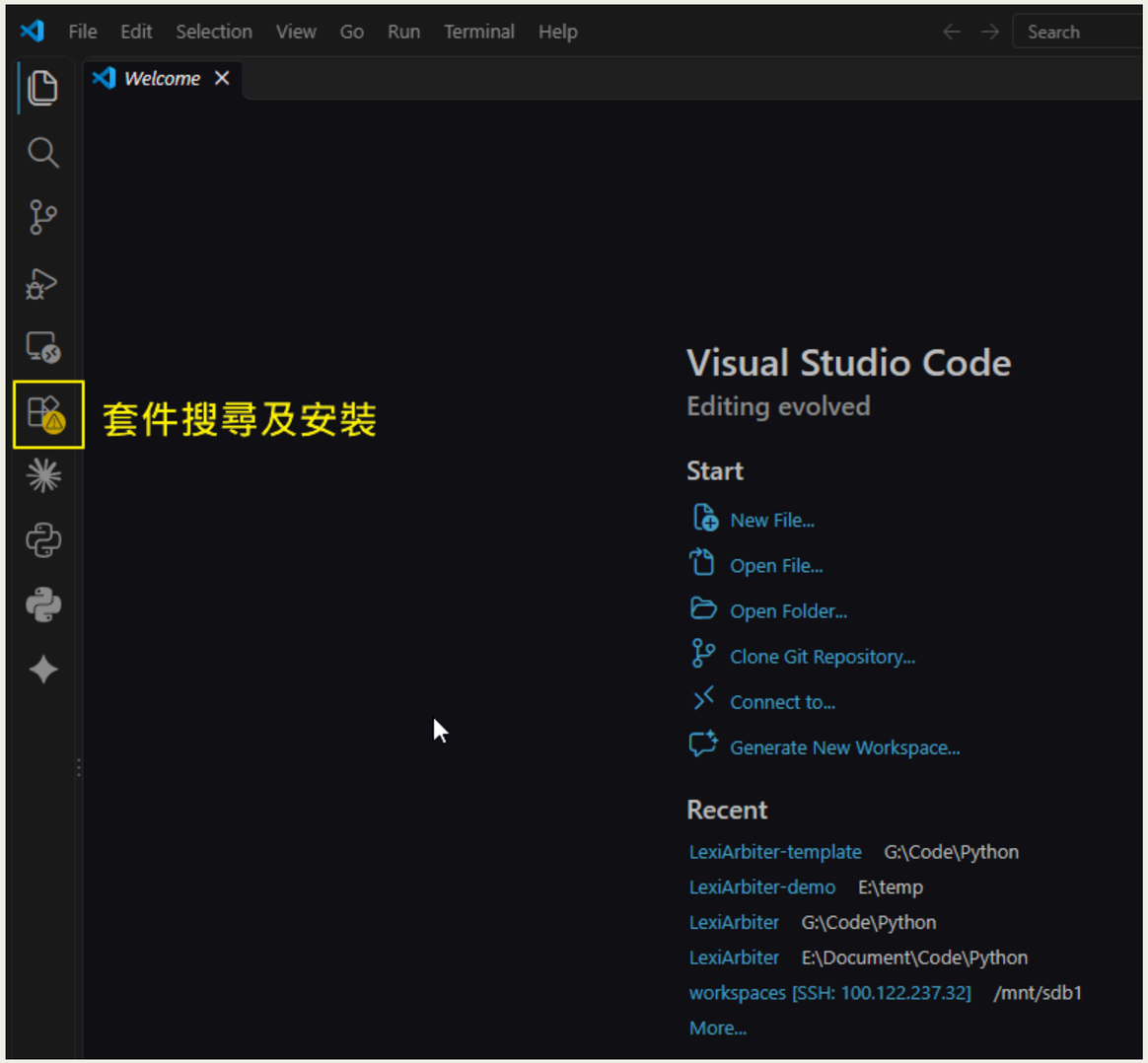
整合在一起，開發時就會比較順手，這種工具統稱 IDE （Integrated Development Environment， 整合開發環境）

其中一個很好用的工具就是 VS Code，免費而且有很多套件支援各種程式語言、各種開發用途。

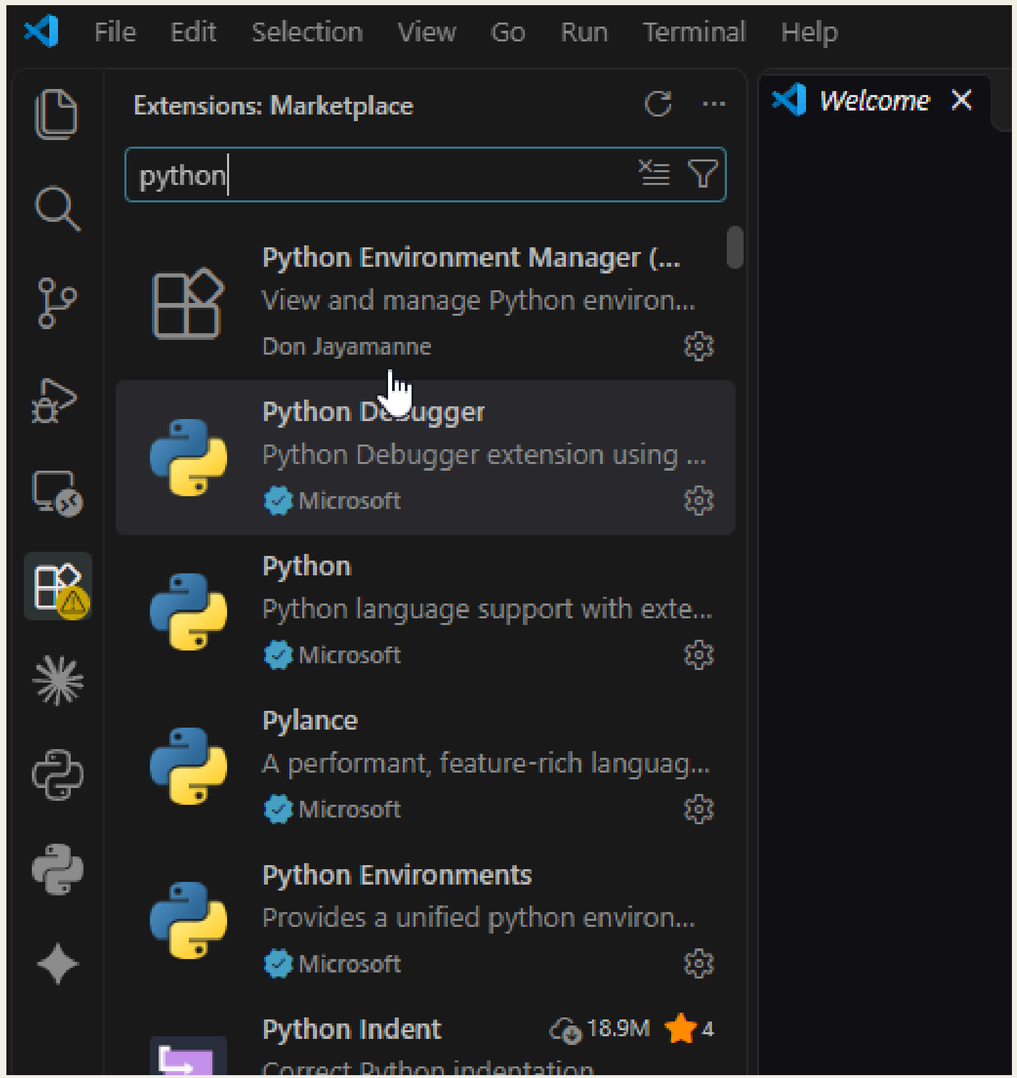


TIPS

vscode 下載安裝完成後打開概會是以下畫面，若要安裝套件可從左邊工具列中尋找套件並安裝。

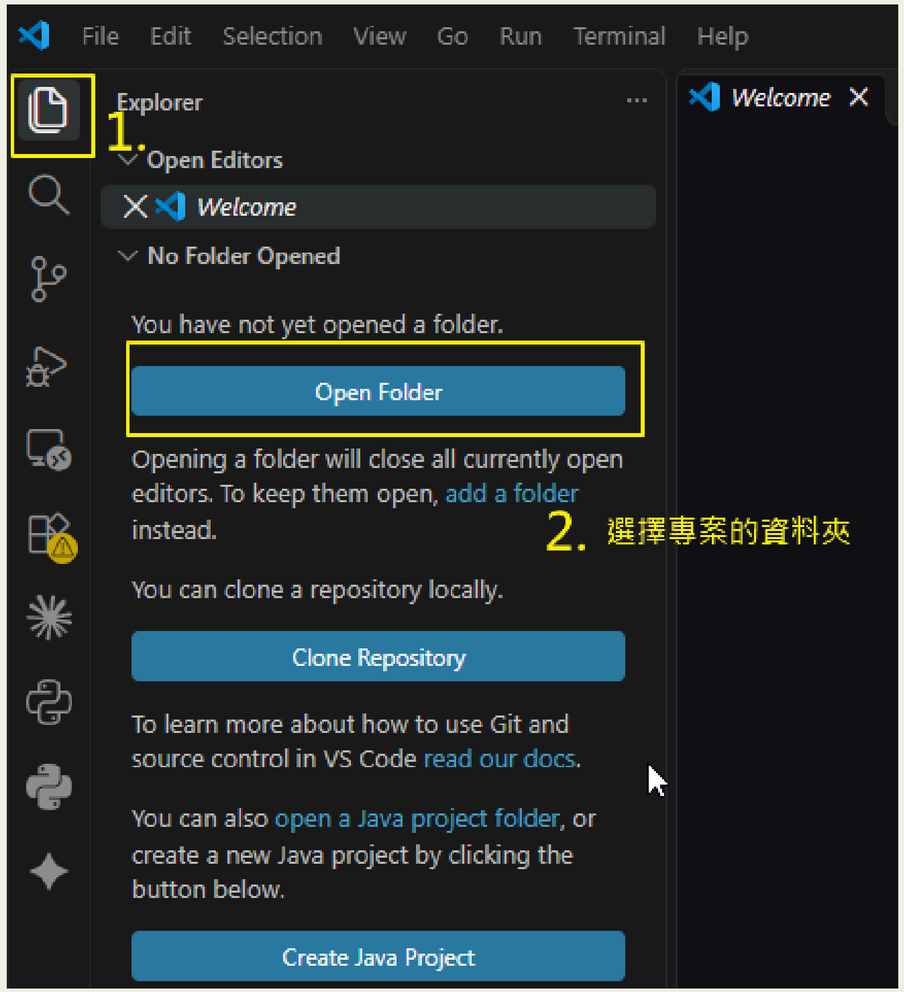


有很多讓 python 開發更方便的套件，可自行研究、安裝。即使不安裝任何套件，vscode也已具備基本的專案內搜尋、基礎語法上色可以直接用。

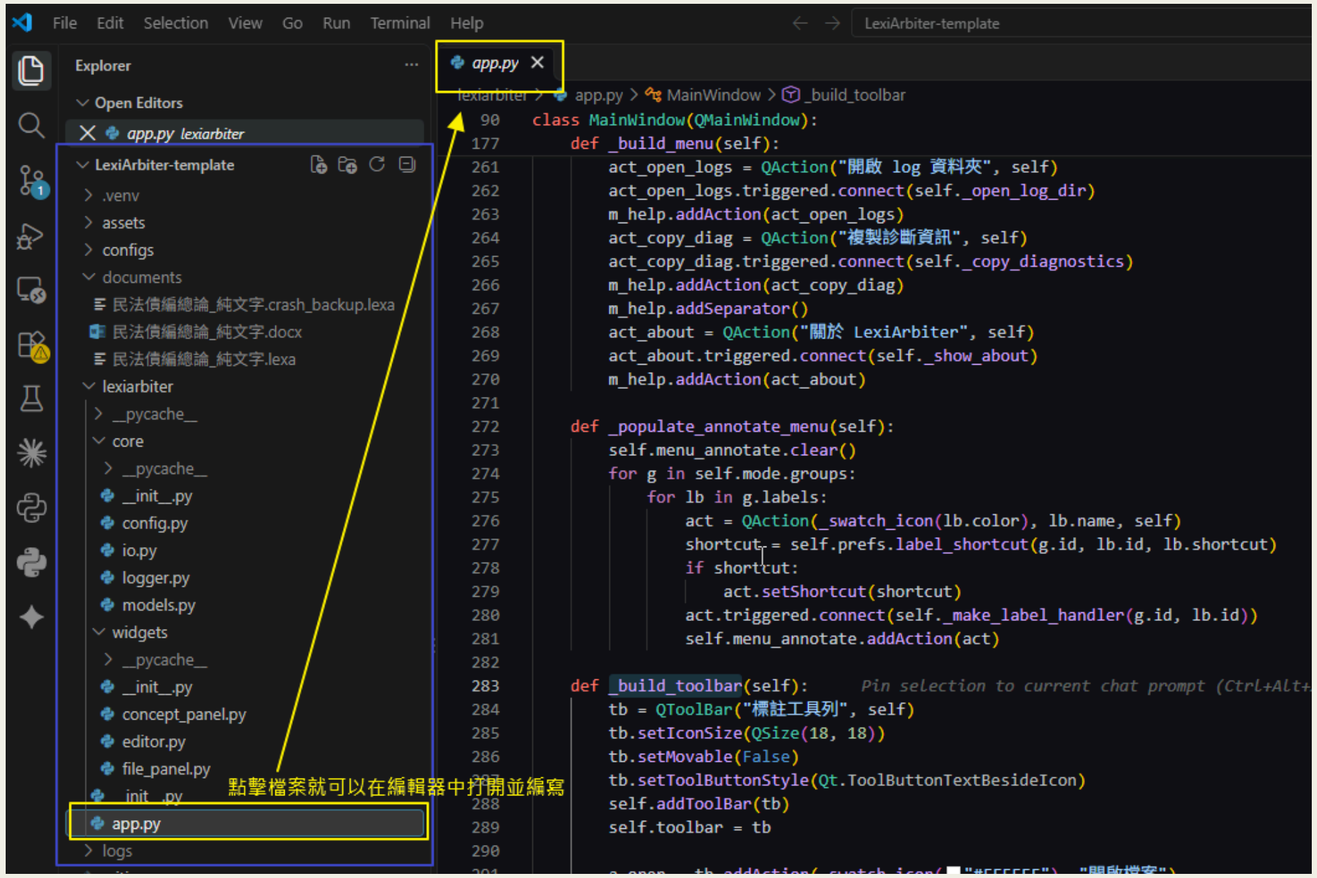


TIPS - 打開專案

從左邊工具列中打開專案資料夾

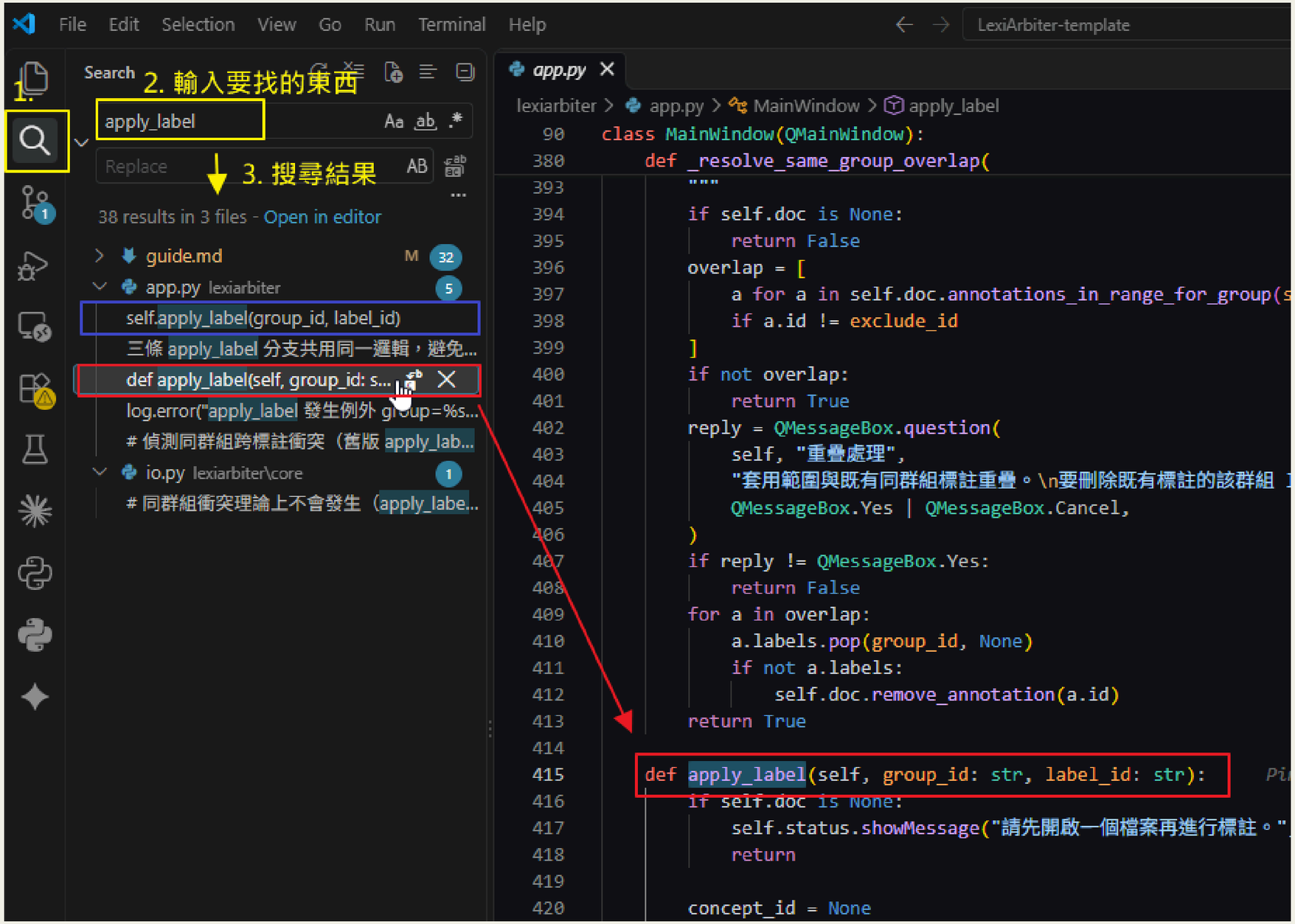


開啟專案資料夾後可以看到藍色框框內就是專案資料夾的結構以及裡面有的檔案，對檔案點擊就能打開對應檔案編輯

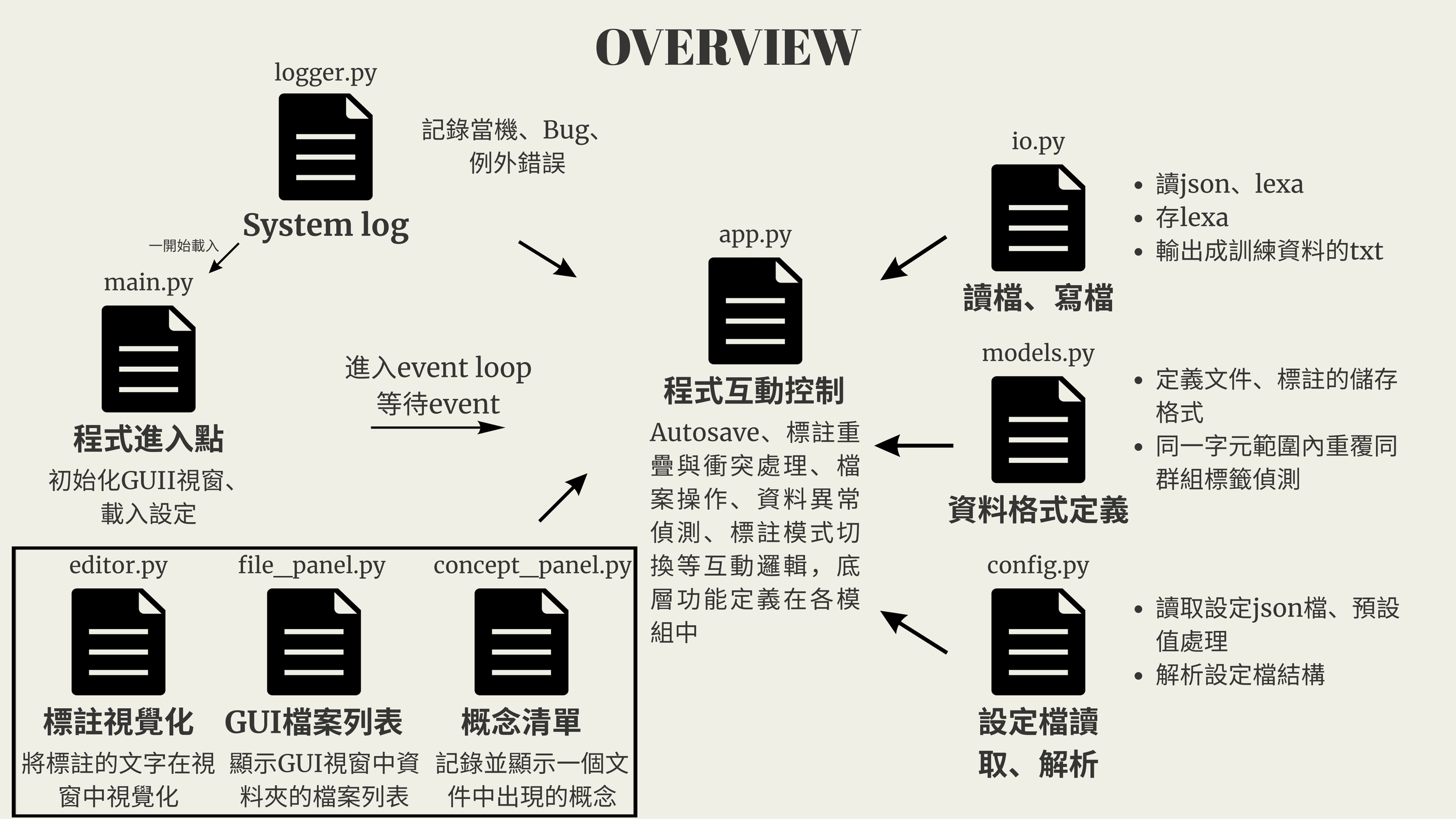


TIPS - 專案內搜尋

- 從左邊工具列中點搜尋並輸入要找的東西 (function、變數等)
- 下方會找出在這個專案資料夾中所有與搜尋字串一樣的文字，並標示位於哪個檔案中，點擊後就會在右邊編輯器中打開該檔案並跳轉到文字所在位置。
- 以右圖為例，找尋 “apply_label” 可以看到紅色框框為 “def” 開頭，表示這是函式定義位置，並且可知是在 app.py 底下，而藍色框框則是其他有呼叫 apply_label的地方，點擊後就可以跳轉到 apply_label 被呼叫的位置。
- 透過這種方式可以比較快的找出某個function被誰、在哪裡被呼叫以及定義在哪裡，把流程線串起來。



OVERVIEW


logger.py
System log
記錄當機、Bug、
例外錯誤
一開始載入
main.py
程式進入點
初始化GUII視窗、
載入設定
進入event loop
等待event
app.py
程式互動控制
Autosave、標註重疊與衝突處理、檔案操作、資料異常偵測、標註模式切換等互動邏輯，底層功能定義在各模組中
io.py
讀檔、寫檔
讀json、lexa
存lexa
輸出成訓練資料的txt
models.py
資料格式定義
定義文件、標註的儲存格式
同一字元範圍內重覆同群組標籤偵測
config.py
設定檔讀取、解析
讀取設定json檔、預設值處理
解析設定檔結構
editor.py
標註視覺化
將標註的文字在視窗中視覺化
file_panel.py
GUI檔案列表
顯示GUI視窗中資料夾的檔案列表
concept_panel.py
概念清單
記錄並顯示一個文件中出現的概念

一個標註的格式定義 model.py

- start (int)：記錄標註在文件中的起始字元位置。
- end (int)：記錄標註在文件中的結束字元位置。
- labels (dict[str, str])：儲存區段所套用的標籤，結構為 group_id 對應到 label_id，處理一段文字需要有兩種不同標籤(label groups)的情況。
- id (str)：標註物件的唯一識別碼，用來精確定位標註區段。
- note (str)：用來記錄附加資訊的文字備註，預設為空字串。
- concept_id (str)：記錄術語對應的概念id
- explanation_start (int): 記錄解釋在文件中的起始位置。
- explanation_end (int): 記錄解釋在文件中的結束位置。
- broader_concept_id (str): 記錄術語對應的上位概念id

```python
@dataclass
class Annotation:
    """A single annotation span on the document.

    `labels` is a mapping from group_id (e.g. "argument_type") to label_id
    (e.g. "major_premise"). A span may have labels from one, several, or all
    groups defined in the active annotation mode (partial annotation is OK).
    """

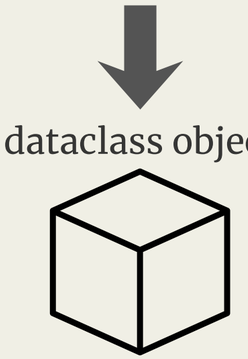
    start: int
    end: int
    labels: dict[str, str] = field(default_factory=dict)
    id: str = field(default_factory=lambda: uuid.uuid4().hex[:12])
    note: str = ""
    concept_id: Optional[str] = None  # 綁定概念 ID
    explanation_start: Optional[int] = None
    explanation_end: Optional[int] = None
    broader_concept_id: Optional[str] = None
```

start / end：術語的開始與結束位置

concept_id：術語對應的概念

explanation_start / explanation_end：解釋的開始與結束位置

broader_concept_id：術語對應的上層概念



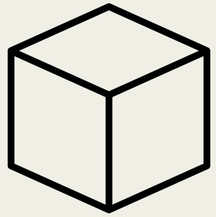
dataclass object

Annotation

表示文件中的一個標註

整個文件的格式定義　model.py

dataclass object



Document

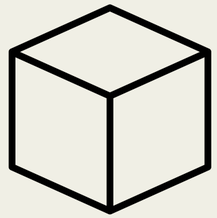
程式介面中打開一個文件時，記錄一個文件的文字內容、metadata、標註、是否修改過且未被存檔(dirty)

- text (str)：這份文件的完整原始純文字內容。
- schema_id (str)：記錄這份文件是用哪一個「標註模式 (Schema)」進行標註的，確保讀取時模式相符。
- source_filename (str)：原始來源檔案的檔名。
- source_meta (dict)：存放原始檔案（例如外部匯入的 JSON）裡附帶的額外資訊（Metadata）。
- concepts (dict): 記錄文件中出現過的概念 {concept_id: concept_name}
- file_path (Optional[str])：目前這個檔案在硬碟中的路徑（也就是載入的 .json 或儲存的 .lexa 的位置）
- dirty (bool)：這是一個非常關鍵的布林值 (Boolean) 旗標，用來標記這份文件「是否被修改過且尚未存檔」。

```python
@dataclass
class Document:
    """In-memory representation of a document being annotated."""

    text: str
    annotations: list[Annotation] = field(default_factory=list)
    schema_id: str = ""
    source_filename: str = ""
    source_meta: dict = field(default_factory=dict)
    concepts: dict[str, str] = field(default_factory=dict)  # 新增
    file_path: Optional[str] = None  # path of the loaded .json or
    dirty: bool = False
```

- dataclass object

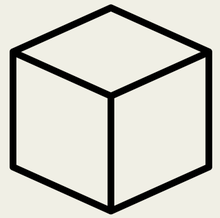


Annotation

表示文件中單一個標註區段

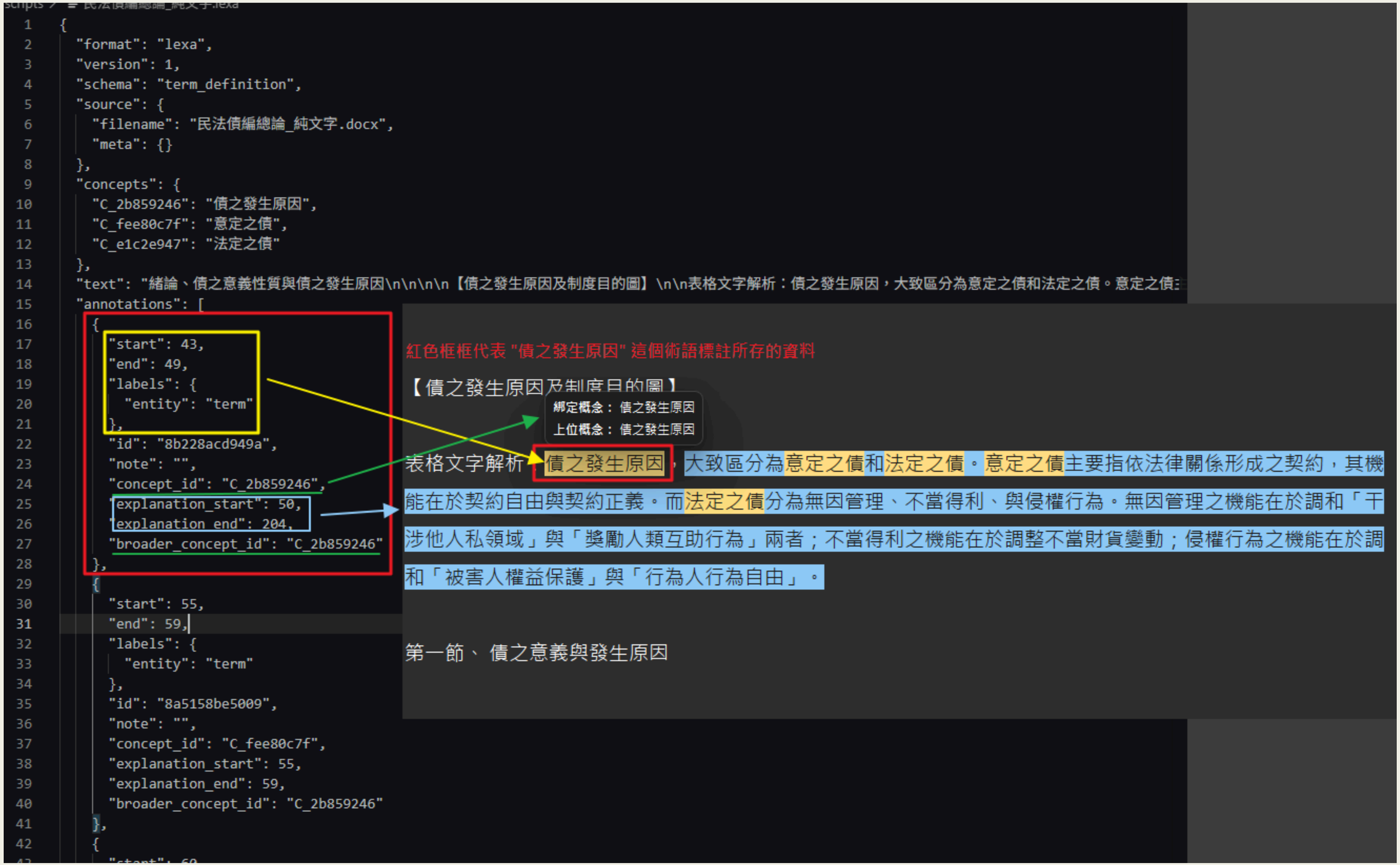
…

dataclass object



Annotation

表示文件中單一個標註區段

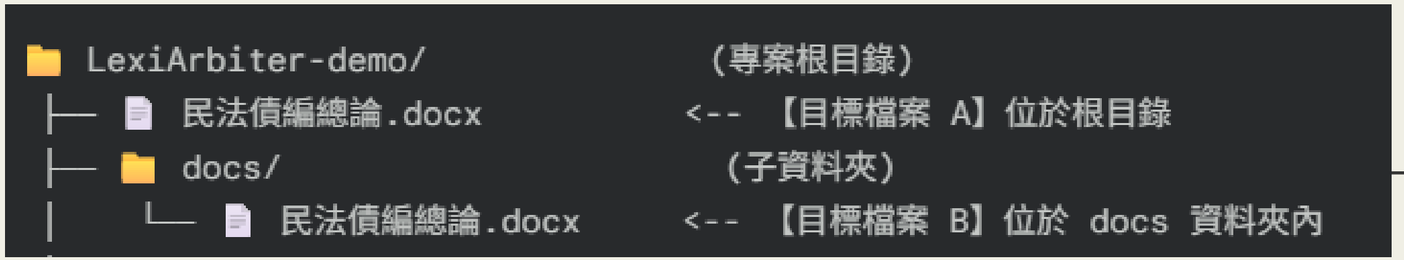

{
"format": "lexa",
"version": 1,
"schema": "term_definition",
"source": {
"filename": "民法債編總論_純文字.docx",
"meta": {}
},
"concepts": {
"C_2b859246": "債之發生原因",
"C_fee80c7f": "意定之債",
"C_e1c2e947": "法定之債"
},
"text": "緒論、債之意義性質與債之發生原因\n\n\n\n【債之發生原因及制度目的圖】 \n\n表格文字解析：債之發生原因，大致區分為意定之債和法定之債。意定之債:
"annotations": [
{
"start": 43,
"end": 49,
"labels": {
"entity": "term"
},
"id": "8b228acd949a",
"note": "",
"concept_id": "C_2b859246",
"explanation_start": 50,
"explanation_end": 204,
"broader_concept_id": "C_2b859246"
},
{
"start": 55,
"end": 59,
"labels": {
"entity": "term"
},
"id": "8a5158be5009",
"note": "",
"concept_id": "C_fee80c7f",
"explanation_start": 55,
"explanation_end": 59,
"broader_concept_id": "C_2b859246"
},
{
紅色框框代表 "債之發生原因" 這個術語標註所存的資料
【債之發生原因及制度目的圖】
綁定概念： 債之發生原因
上位概念： 債之發生原因
表格文字解析 債之發生原因，大致區分為意定之債和法定之債。意定之債主要指依法律關係形成之契約，其機能在於契約自由與契約正義。而法定之債分為無因管理、不當得利、與侵權行為。無因管理之機能在於調和「干涉他人私領域」與「獎勵人類互助行為」兩者；不當得利之機能在於調整不當財貨變動；侵權行為之機能在於調和「被害人權益保護」與「行為人行為自由」。
第一節、 債之意義與發生原因

檔案路徑　io.py

什麼是路徑?

把電腦裡的資料夾想像成真實世界的建築物:

- 絕對路徑（Absolute Path）：就像是完整的郵遞區號與地址。無論你現在身在何處，只要照著這個地址，就一定能找到該地點。例如:
  - Windows：“E:\LexiArbiter-demo\民法債編總論.docx”
  - Mac/Linux：“/Users/LexiArbiter-demo/民法債編總論.docx”
- 相對路徑（Relative Path）：就像是向路人問路（例如：出大門後右轉）。取決於你「現在站在哪裡」，如果起點不同，同樣的走法就會到達錯誤的地方。表示法:
  - “.” 表示目前所在的資料夾。例如："民法債編總論.docx" 或 "./民法債編總論.docx" 表示在目前的資料夾底下找 “民法債編總論.docx”。
  - “..” 表示上一層資料夾。例如："../民法債編總論.docx"：到上一層資料夾找 民法債編總論.docx。
  - 而"docs/民法債編總論.docx"：進入目前的資料夾底下的 docs 資料夾，尋找 民法債編總論.docx



可能會用到或要改的地方：操作流程相關

建議不熟 GUI 程式的人先去看前面的 GUI 101 章節

1.1 編輯器拖選文字：流程 A：放開滑鼠（拖選結束）時跑一次

按下左鍵：AnnotationEditor.mousePressEvent(ev)
(Qt負責偵測，記下「左鍵拖曳開始了」)

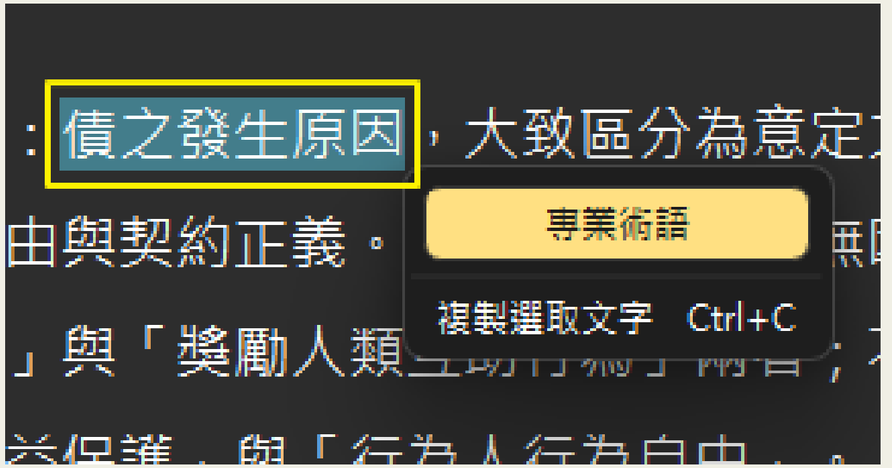
⬇

放開左鍵： AnnotationEditor.mouseReleaseEvent(ev)
(Qt負責偵測)

⬇

呼叫 self.has_selection() → True / False
(檢查有沒有選到字)

⬇



有選到字（True）：

- emit selection_finished(滑鼠放開位置的螢幕座標)。
- 接到MainWindow._show_quick_label_popup(global_pos)
  (見1.4節)。

註：在template-v1.1中，selection_finished 只有在 popup 模式（auto_popup_on_selection: true）時，才會在 MainWindow.___init___ 接到 _show_quick_label_popup。工具列模式下，流程 A 走到 emit 就停了，沒有任何函式被呼叫。

1.2 編輯器拖選文字：流程 B：選取範圍只要有變動就跑（拖曳過程中會一直跑，鍵盤選取也會跑）

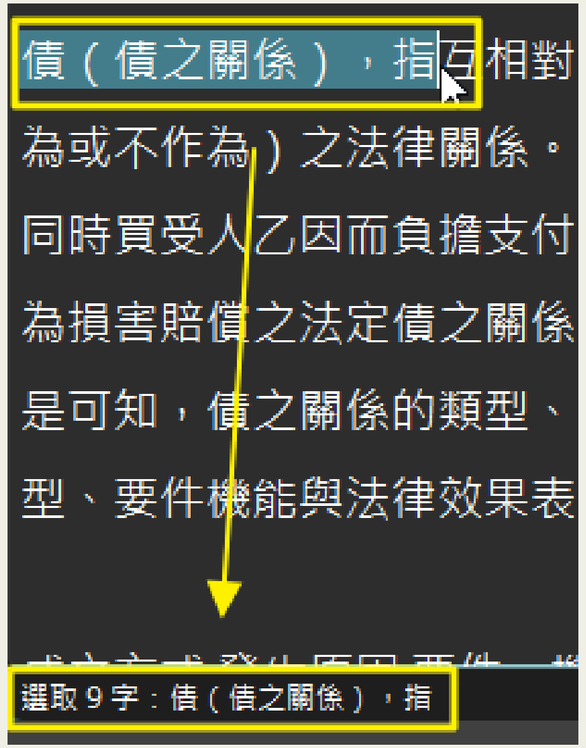
選取範圍改變：AnnotationEditor._on_selection_changed()
(Qt負責偵測)

⬇

呼叫 self.storage_selection() → (s, e) (送出新的選取範圍)

⬇

MainWindow._on_selection_changed(s, e) (在狀態列顯示「選取 N 字：...」)
※ 範圍已經透過參數 s, e 傳進來了，這裡不需要再呼叫 storage_selection()
※ 函式內用 self.doc.text[s:e] 取出文字



1.3 選取文字相關FUNCTION用法

- 任何地方，需要知道選取範圍：s, e = self.editor.storage_selection()，回傳(start, end)；沒選取時 s == e。
- 任何地方，只要知道有沒有選：self.editor.has_selection()，回傳 True / False。
- 在接到 selection_changed 的函式裡（例如 MainWindow._on_selection_changed）：直接用參數 s, e不用另外呼叫
- 在接到 selection_finished 的函式裡（例如 _show_quick_label_popup）：自己呼叫 storage_selection()，
- 需要選取起來的具體文字內容：self.doc.text[s:e] (記得先用 storage_selection() 取得 s, e)。

註：範圍一律用 storage_selection() 取得，不要用 textCursor().selectionStart()，否則遇到有換行的文件，位置會偏掉。

1.4 按下 POPUP 選單裡的標籤按鈕

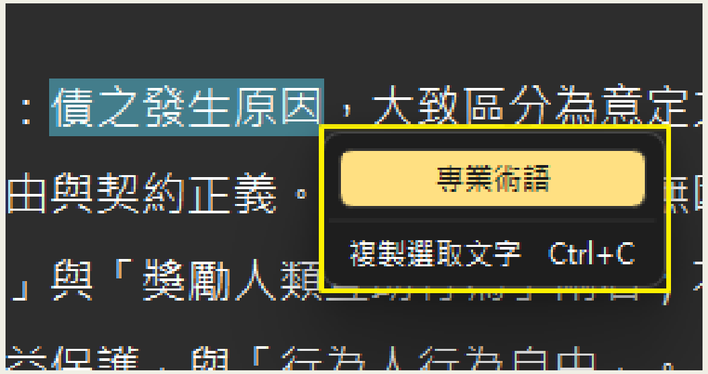
MainWindow._show_quick_label_popup(global_pos)。

⬇

建立 QMenu，呼叫 self._make_quick_label_widget(menu)
（依標註模式的標籤自動產生一排標籤按鈕並回傳）。

⬇

加入「複製選取文字」項目（按下 → self.editor.copy() ）。



⬇

menu.exec(...) （顯示選單；選單關閉之前，程式會停在這一行）。

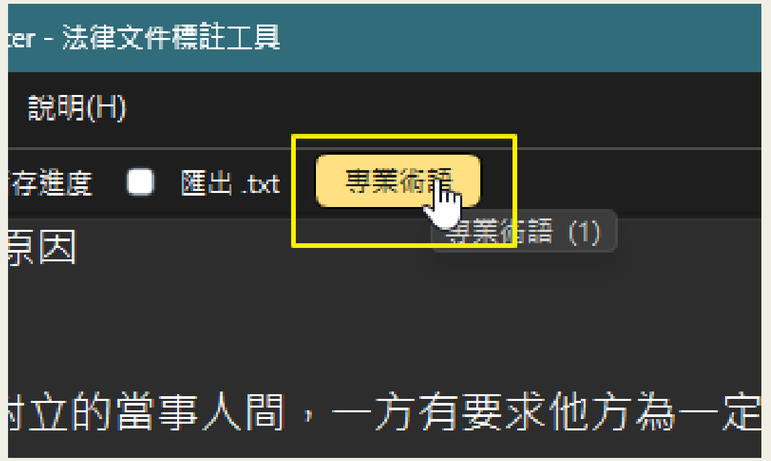
使用者按下某個標籤按鈕：
→ **_make_label_handler(group_id, label_id)** 連結到apply_label處理標註行為。
→ **MainWindow.apply_label(group_id, label_id)（標註重點，見 1-)**。
→ menu.close()

註：configs/user_preferences.json 中 auto_popup_on_selection 為 true 時popup才會出現，否則顯示上方工具列

1.5 工具列的標籤按鈕

程式啟動時：
MainWindow.init → MainWindow._build_toolbar()

MainWindow._populate_label_buttons()



使用者按下某個標籤按鈕：
→ **_make_label_handler(group_id, label_id)** 連結到apply_label處理標註行為。
→ **MainWindow.apply_label(group_id, label_id)（標註重點，見 1- ）**。

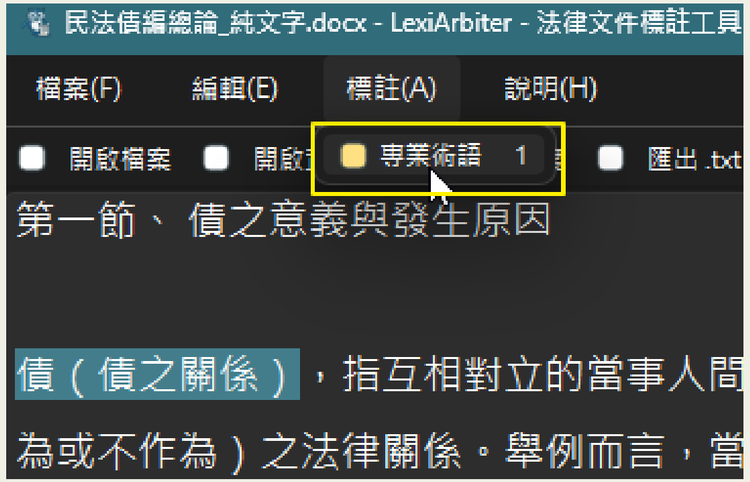
註：auto_popup_on_selection 為 false 時才會出現

1.6 視窗頂端選單列或標籤快捷鍵

程式啟動時：
MainWindow._build_menu()

⬇

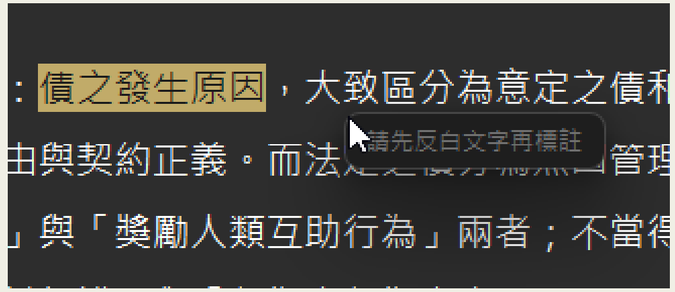
MainWindow._populate_annotate_menu()
（把每個標籤加進「標註」選單，有設定快捷鍵的一併設定）

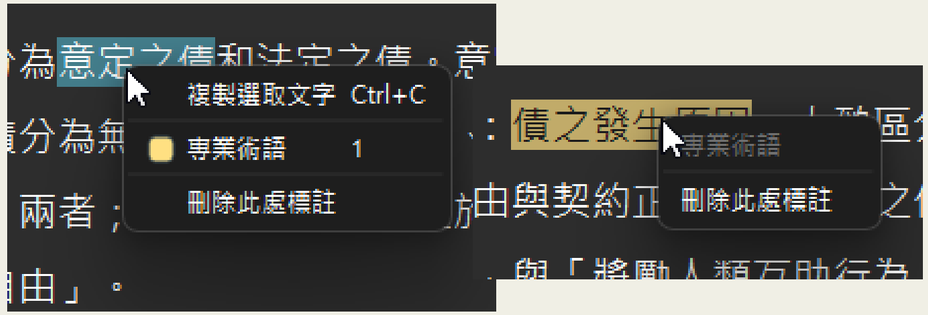


使用者按下某個標籤按鈕：
→ **_make_label_handler(group_id, label_id)** 連結到apply_label處理標註行為。
→ **MainWindow.apply_label(group_id, label_id)（標註重點，見 1- ）。**

註：auto_popup_on_selection 為 false 時才會出現

1.7 編輯器上按右鍵





按右鍵：AnnotationEditor._on_context_menu(pos)（pos是右鍵點擊的位置，由Qt偵測並回傳）

↓

呼叫 self.cursorForPosition(pos) → 得到右鍵位置的游標 cursor

↓

沒有選取文字時：呼叫 self.setTextCursor(cursor)，把文字游標移到右鍵位置（有選取時則不動，以免清掉反白）

↓

呼叫 self._omap.to_storage(cursor.position()) 得到點擊位置在原文中的位置

↓

呼叫 self.storage_selection() (參考1.3節) → 呼叫 self._doc.annotations_at(位置) 看點擊的位置上有沒有標註

↓

把剛剛查的資訊傳給 **MainWindow._build_context_menu** 根據不同情況建立選單，可從這裡修改選單的內容

↓

呼叫 self.doc.find_annotation(ann_id)（用來顯示選單中選項的標籤名稱，以及略過它已經套用的標籤） → 顯示選單

1.7 編輯器上按右鍵

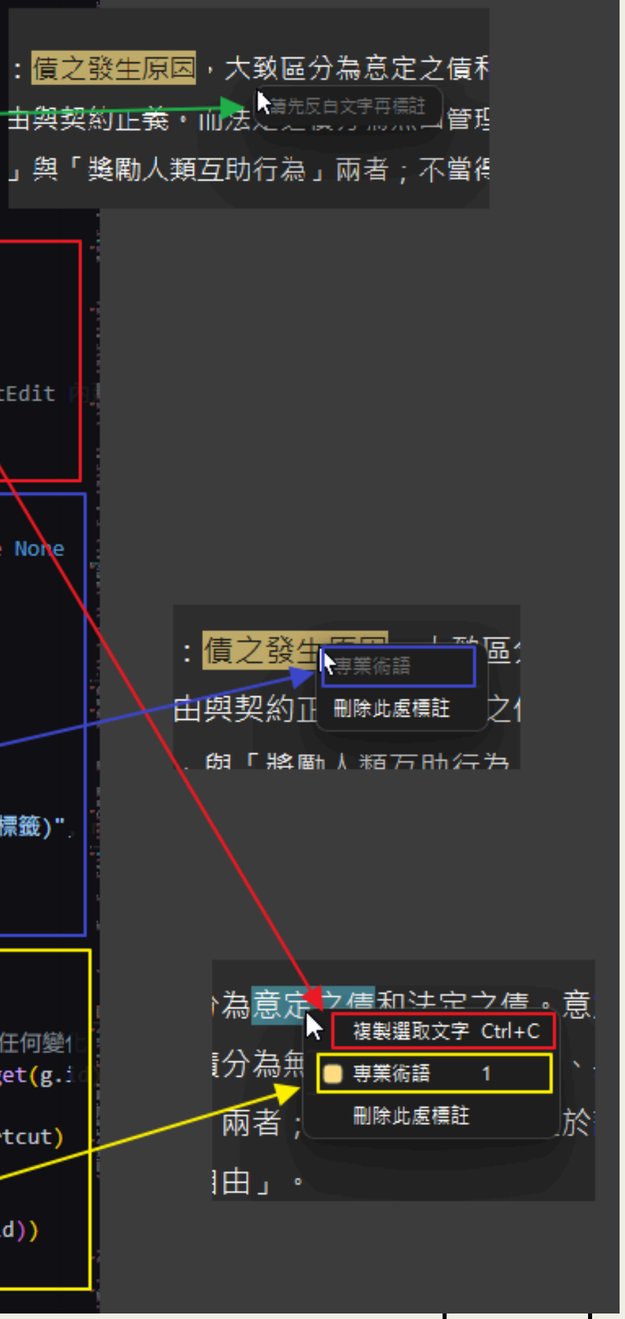
```python
class MainWindow(QMainWindow):
    def _build_context_menu(self, menu: QMenu, ann_id: Optional[str],
        has_selection = sel_end > sel_start

        # 沒有反白、右鍵處也沒有既有標註時，標籤點了也沒效果，只顯示提示
        if not has_selection and ann_id is None:
            hint = QAction("請先反白文字再標註", menu)
            hint.setEnabled(False)
            menu.addAction(hint)
            return

        # 如果有選取文字，在選單最上方加入「複製」選項
        if has_selection:
            act_copy = QAction("複製選取文字", menu)
            act_copy.setShortcut("Ctrl+C")  # 提示使用者亦可用快捷鍵
            act_copy.triggered.connect(self.editor.copy)  # 直接呼叫 QTextEdit
            menu.addAction(act_copy)
            menu.addSeparator()

        # If hovering over an annotation, show its info.
        ann = self.doc.find_annotation(ann_id) if ann_id is not None else None
        if ann is not None:
            lbl_lines = []
            for g in self.mode.groups:
                lid = ann.labels.get(g.id)
                if lid:
                    lb = g.label(lid)
                    if lb:
                        lbl_lines.append(lb.name)
            head = QAction("、".join(lbl_lines) if lbl_lines else "(無標籤)",
            head.setEnabled(False)
            menu.addAction(head)
            menu.addSeparator()

        for g in self.mode.groups:
            for lb in g.labels:
                # 未反白時只能改右鍵處的標註；已套用的同一個標籤再套一次不會有任何變
                if not has_selection and ann is not None and ann.labels.get(g.
                    continue
                shortcut = self.prefs.label_shortcut(g.id, lb.id, lb.shortcut)
                text = lb.name + (f"\t{shortcut}" if shortcut else "")
                act = QAction(_swatch_icon(lb.color), text, menu)
                act.triggered.connect(self._make_label_handler(g.id, lb.id))
                menu.addAction(act)
```



1.8 標註行為：MainWindow.apply_label()

術語模式下才會執行：(打開concept panel)

呼叫 self.editor.has_selection() (見1.3)

呼叫 self.editor.textCursor().selectedText() → 取得選取的文字（拿來當concept panel的預設值）

dlg = ConceptSelectionDialog(self.doc.concepts, default_text=選取文字, parent=self) 建立concept panel，並用dlg.exec()開啟視窗。(參考2.5)

dlg.get_result() 得到使用者在concept panel選的結果。

分三種情況決定按下標註鈕後的行為：

- 情況 A：沒有選取文字 → 修改游標所在的那個標註 (※ 情況 A 因為標註設定檔中只有「專業術語」一個標籤，對已標註文字再套一次「專業術語」等於沒改，所以看起來沒反應。新增第二個 後才會真的改到東西。)
- 情況 B：有選取文字且剛好有一個既有標註的範圍等於 (s, e) → 修改那個標註
- 情況 C：其他情況 → 建立新標註

接下一頁

1.8 標註行為：MainWindow.apply_label()

情況 C：建立新標註的流程

呼叫 self._resolve_same_group_overlap 檢查重疊並處理衝突

※ 沒有重疊：回傳 True，繼續建立；有重疊：跳出重疊處理對話框。
※ 內部呼叫 **self.doc.annotations_in_range_for_group 找出衝突的標註 (待完成)**

新增一個標註物件 ann = Annotation(start=s, end=e, labels={group_id: label_id}, concept_id=...)

※ 用來存一個標註的資訊 (參考“一個標註的格式定義”、model.py)

呼叫 self.doc.add_annotation(ann) 將標註存到記憶體裡

收尾四件套：

- self.doc.dirty = True → 標題列出現 *、關閉時提醒存檔、自動存檔才會存
- self.editor.refresh_highlights() → 重畫標註
- self._refresh_status() → 更新狀態列統計
- self._update_window_title() → 更新標題列

※ 有改到概念時需要 call： self.concept_panel.refresh(self.doc)

1.9 重疊偵測 model.py

參考1.8標註行為

- Annotation.overlaps()：比對兩個annatation物件的結束位置與起始位置是否完全錯開。

- Document.annotations_in_range()：找出與指定的字元範圍（從start到end）有任何重疊的所有標註物件。當使用者選取了一段文字並執行「刪除游標處標註」或「清除選取段」時，程式就會呼叫這個方法，一口氣把這段範圍內所有牽涉到的標註物件找出來並進行刪除或清除。

- Document.annotations_in_range_for_group()：找出與指定範圍重疊的標註，並進一步過濾只保留含有特定群組（group_id）標籤的標註物件。當使用者要在一段文字上標記「群組 A」的新標籤時，系統必須確認「這段範圍內，是不是已經有其他標註也佔用了『群組 A』？」。這時候就會呼叫這個方法，精準找出同範圍內發生群組衝突的舊標註，進而跳出警告視窗詢問使用者是否要刪除舊標籤再套用。

1.9 重疊偵測 model.py

參考1.8標註行為

呼叫function檢查有沒有重疊，並依情況提示使用者

In app.py

```python
def _resolve_same_group_overlap(
    self, s: int, e: int, group_id: str,
    exclude_id: Optional[str] = None,
) -> bool:
    """檢查 [s, e) 範圍內除了 exclude_id 以外，是否有其他標註已佔用 group_id。

    有重疊就彈窗詢問是否刪除既有的同群組 label：
    - 使用者同意 → 就地清掉（若清空整條 ann 的 labels 則整條移除），回 True。
    - 使用者取消 → 回 False，呼叫端應 return 不寫入。
    無重疊直接回 True，呼叫端繼續即可。

    三條 apply_label 分支共用同一邏輯，避免「exact match / cursor 內套用」
    路徑繞過跨標註同群組衝突檢查（會在匯出時切碎成奇怪段落）。
    """
    if self.doc is None:
        return False
    overlap = [
        a for a in self.doc.annotations_in_range_for_group(s, e, group_id)
        if a.id != exclude_id
    ]
    if not overlap:
        return True
    reply = QMessageBox.question(
```

呼叫負責檢查的function

In model.py

```python
def annotations_in_range_for_group(
    self, start: int, end: int, group_id: str
) -> list[Annotation]:

    result = []

    for a in self.annotations:
        # TODO: 完成以下缺失的部份
        # 請判斷既有標註是否與使用者選取的範圍 (start, end) 有重疊？
        # 如果有重疊，請將 is_overlapping 設為 True；否則為 False。

        is_overlapping = False  # <--- 請修改這行的邏輯

        # ============================================================

        has_group = (group_id in a.labels)

        if is_overlapping and has_group:
            result.append(a)

    # 把收集到的所有衝突標註回傳出去
    return result
```

1.10 用左鍵點擊已標註的文字

※ 目前點下去沒有任何反應。 AnnotationEditor 裡已經宣告了 annotation_clicked = Signal(str)，但沒有任何地方 emit 它，也沒有任何函式 connect 它。你需要自己把下面這條流程接起來：

放開左鍵：AnnotationEditor.mouseReleaseEvent(ev)

⬇

呼叫 self.has_selection()（確認只是單純點一下，沒有拖選）

⬇

呼叫 self.cursorForPosition(ev.position().toPoint()) 再呼叫 self._omap.to_storage(...) 取得點擊的位置

⬇

呼叫 self._doc.annotations_at(點擊位置) 取得標註 list

⬇

有標註：⋯› emit annotation_clicked (發射 ann_id 作為Signal)
⋯› 接到 MainWindow 裡你寫的函式(也就是收到ann_id Signal時要處理的函式)

呼叫 self.doc.find_annotation(ann_id) 取得 Annotation 物件

⬇

（顯示資訊、highlight 解釋段落等...見 2-8、2-9）

Hint：

- 找標註的那三個呼叫，和 1.7 AnnotationEditor._on_context_menu 找 ann_id 的方式完全一樣；有多筆標註時，同樣取最後一筆。
- 在 AnnotationEditor 裡，文件是 self._doc，不是 self.doc；還沒開檔時它是 None，要先擋掉。
- emit、connect、接收函式的完整寫法，可以參考 2-7（概念清單點擊），兩者是同一個模式。

1.11 滑鼠停在標註上（TOOLTIP）

滑鼠停在編輯區一下下：AnnotationEditor.viewportEvent(ev) (in editor.py)

⬇

呼叫 self._display_pos_under_mouse(ev.pos()) 取得滑鼠下那個字的位置

⬇

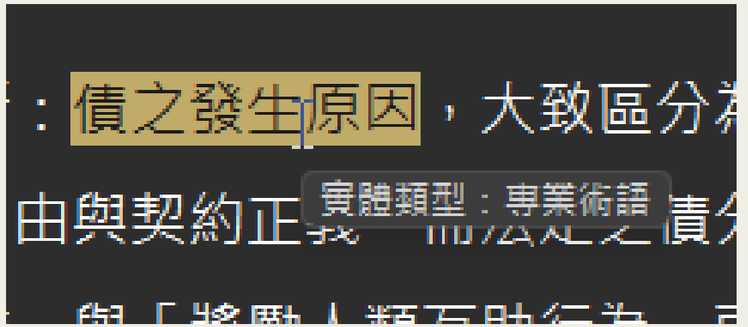
self._doc.annotations_at(storage 位置) → 蓋住這個字的標註

⬇

QToolTip.showText(..., self._tooltip_text(ann)) ← tooltip 的內容在這裡決定

⬇

懸浮提示顯示中、滑鼠繼續移動：
AnnotationEditor.mouseMoveEvent(ev) 依情況做不同反應。



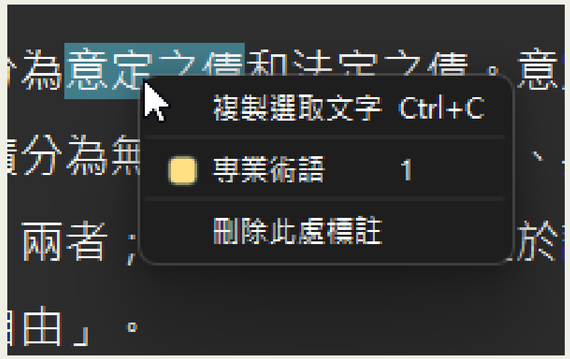
可能會用到或要改的地方：GUI介面相關

想修改 GUI 介面的話 template 裡幾乎都有一樣或很像的寫法。以下會指出去哪裡參考，請先找到那段程式碼讀懂，再仿照它寫出你自己的版本。

建議不熟 GUI 程式的人先去看前面的 GUI 101 章節

2.1 修改右鍵選單

產生位置： MainWindow._build_context_menu(menu, ann_id, sel_start, sel_end)
(in app.py，可一併參考 1.7 節)



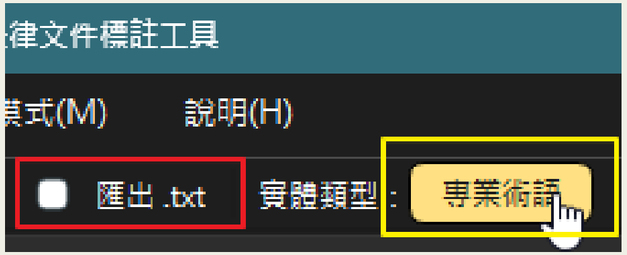
Hint:

- 函式中段已經用 ann_id 取得 ann（右鍵點到的標註，沒點到就是 None）。想要「只在點到標註時出現」，就把程式碼加在它後面、用 ann 判斷；想「只在術語模式出現」，就判斷 self.mode.id（apply_label 開頭有一樣的判斷）。
- 分隔線、灰色不能按的項目、快捷鍵提示，這個函式裡都有現成的例子：menu.addSeparator()、hint.setEnabled(False)、act_copy.setShortcut(...)。子選單可以參考 _build_menu 裡的 addMenu。
- 在右鍵選單上 setShortcut 只會顯示提示文字，不會變成真的快捷鍵（真的快捷鍵見 1.6、2.3、2.10）。
- 按下去時要把 ann 傳給你的函式，參考 _build_menu 。如果是在 for 迴圈裡建立多個項目，參考 _make_quick_label_widget 。

2.2 在工具列加一個按鈕

產生位置： MainWindow._build_toolbar（一般按鈕，右圖紅框），以及 MainWindow._populate_label_buttons（標籤按鈕，右圖黃框）

※ 按鈕為什麼按了會呼叫你的函式，參考GUI 101 中 0.3 節。



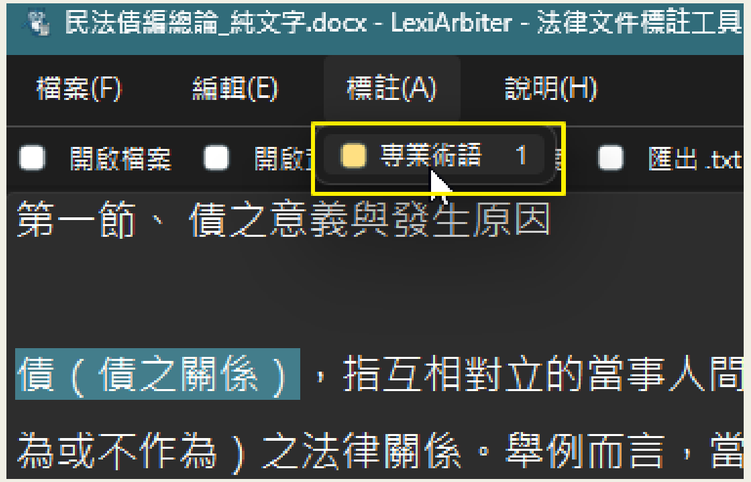
Hint：

- 一般按鈕（兩種模式下都會出現）：參考 _build_toolbar 。
- 彩色按鈕：參考 _populate_label_buttons 迴圈裡建立按鈕的寫法（QPushButton → setStyleSheet → clicked.connect → self.toolbar.addWidget），樣式字串可以參考 _button_style_for。注意 QPushButton 的 signal 是 clicked，不是 triggered。
- 標籤按鈕不用手動加：它們是 _populate_label_buttons 依照標註模式 JSON 自動產生的（見 1.5、2.5、3.1），而且只在 auto_popup_on_selection: false 時出現。
- 想之後把按鈕藏起來或顯示出來：把 addAction / addWidget 回傳的物件存在 self 上，再呼叫 .setVisible(...)。參考MainWindow.___init___ 裡的 self.concept_panel.setVisible。

2.3 在選單列加一個項目（含快捷鍵）

產生位置： MainWindow._build_menu

※ 參考1.6



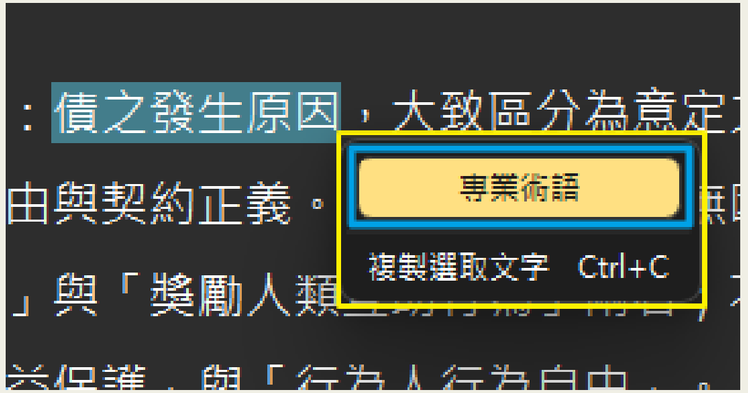
Hint：

- 快捷鍵用 self.prefs.app_shortcut(名稱, 預設值) 取得，使用者就能在設定檔覆寫。加在選單列上的快捷鍵，在整個視窗裡都有效。
- 要放進哪個選單，就 addAction 到那個選單：m_file、m_edit 是現有的選單；想開一個新的選單，參考同一個函式裡的 mb.addMenu(...)。
- 如果這個項目只在開檔後才能用，參考 MainWindow._update_actions，它負責在沒開檔時把項目變灰。

2.4 在 POPUP 選單加按鈕或項目

產生位置： MainWindow._show_quick_label_popup（選單本體，右圖黃框）、MainWindow._make_quick_label_widget（那一排標籤按鈕，右圖藍框）

※ 參考1.4



Hint：

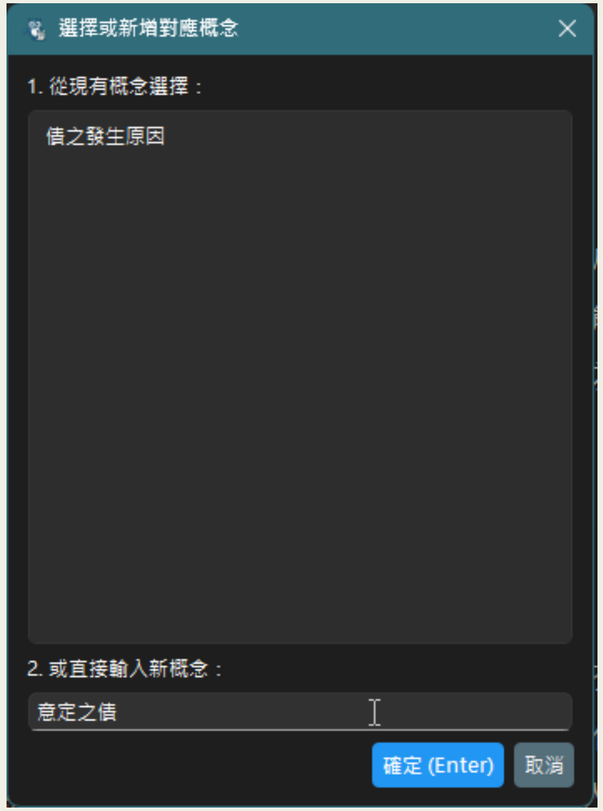
- 加一般選單項目：在 _show_quick_label_popup 用 QAction + menu.addAction(...)。
- 在標籤那一排加按鈕：在 _make_quick_label_widget 裡建立 QPushButton，再 row.addWidget(btn)。按下後記得呼叫 host_menu.close() 關閉選單。

2.5 概念對話框不夠用時怎麼改

位置： widgets/concept_panel.py → ConceptSelectionDialog
唯一的呼叫端： MainWindow.apply_label

ConceptSelectionDialog.___init___(concepts, default_text, parent)
  layout = QVBoxLayout(self)　　← 元件依 addWidget 的順序由上往下排列
- get_result() → (既有id, None) / (None, 新名稱) / (None, None)　← 只讀 self.input_edit

Hint：
- 想多一個欄位：___init___：QLabel 標題 → list_widget 列出既有概念，並用 _concept_map 記住「名稱 → id」→ input_edit 讓使用者輸入 → 最後用 itemClicked 把點到的項目填進輸入框。
- 想在對話框裡顯示更多資訊：參考 default_text 的做法。
- 想在按「確定」時檢查輸入：btn_ok 按下去呼叫的是 self.accept（QDialog 內建的方法），所以要在類別裡覆寫 accept，參考 MainWindow.closeEvent、MainWindow._warn_if_group_conflicts 等。
- 想多一個按鈕：參考 btn_cancel，另外exec() 只會回傳 True（accept）或 False（reject），分不出「取消」和「回上一步」。可以在按鈕的處理函式裡先設一個屬性當作記號，再呼叫 self.reject() 關閉對話框；呼叫端在 dlg.exec() 之後讀這個屬性，決定下一步要做什麼。



2.6 概念清單（右下角側欄）

位置： 位置： widgets/concept_panel.py → ConceptSidebar



- 清單內容在 refresh(doc) 裡產生，每次呼叫都會清空後重新列出 doc.concepts。
- concept_clicked = Signal(str) 已經宣告，但還沒接上任何東西。
- 想讓「點擊概念」有反應，參考 widgets/file_panel.py → FilePanel。它是同一個模式的完整範例：點清單項目 → 發出帶資料的 signal → MainWindow 接起來。

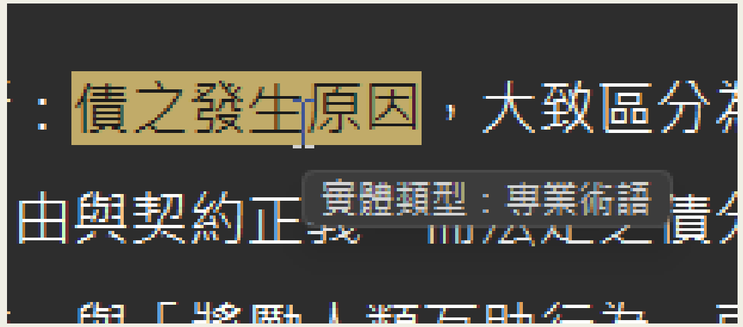
要做的事	FilePanel 裡對應的地方
讓每個項目記住自己的 id	refresh 裡用 QListWidgetItem 建立項目，再用 setData(Qt.UserRole, ...) 存資料
點擊時取出 id、發出 signal	__init__ 把清單的 signal 接到 _on_item_activated，它用 item.data(Qt.UserRole) 取出資料後 emit
在 MainWindow 接到你的函式	MainWindow.__init__ 裡 self.file_panel.file_open_requested.connect(...) 那行

- FilePanel 接的是 itemActivated（雙擊或按 Enter 才觸發）；想要單擊就有反應，改用 itemClicked（ConceptSelectionDialog 裡有用到）。
- 記得在 concept_panel.py 開頭 import QListWidgetItem。

2.7 顯示一個小資訊視窗

AnnotationEditor.viewportEvent 裡的 QToolTip.showText(...)。想顯示在滑鼠目前的位置，可以查 QCursor.pos()

※ 參考1.11



Hint:

- 想自訂內容、點外面會自動關閉： 參考 MainWindow.__show_quick_label_popup。它用 QMenu 當外框，再用 QWidgetAction 把自己做的元件（_make_quick_label_widget 回傳的 QFrame）塞進去，最後用 menu.exec(位置) 顯示。把裡面的按鈕換成 QLabel，就是一個資訊視窗。
- 想把視窗放在「某個標註」旁邊，而不是滑鼠旁邊：參考 AnnotationEditor.jump_to_annotation 的寫法（ann.start 要先經過 _omap.to_display 轉換）。
- cursorRect 回傳的是編輯區（viewport）裡的座標，exec / showText 要的是螢幕座標，要再用 viewport().mapToGlobal(...) 轉換。

2.7 在編輯器上畫額外的 HIGHLIGHTS

位置： AnnotationEditor.refresh_highlights

※ 參考 1.8 apply_label 的收尾四件套

Hint：

- refresh_highlights 每次都會重新產生所有高亮，然後用 self.setExtraSelections(selections) 一次套用。
- 想多畫一段，可以拆成三步：
  - 記住要畫哪一段：參考 _tooltip_ann_ids
  - 畫出來：參考 for ann in … 迴圈裡的東西，顏色的設定方法參考 _apply_format_for_labels 最後的 fmt.setBackground(…)
  - 讓 MainWindow 能開關它：在 AnnotationEditor 寫方法去改那個屬性，改完呼叫 self.refresh_highlights() 重畫（attach 的結尾也是這樣做）。MainWindow 再透過 self.editor.你的方法(…) 呼叫。
- 範圍存的是 storage offset，畫之前要用 _omap.to_display 轉換
- selections 列表中越後面的項目會畫在越上層。
- 不要在其他地方另外呼叫 setExtraSelections，否則下次 refresh_highlights() 時會被蓋掉。

其他

建議不熟 GUI 程式的人先去看前面的 GUI 101 章節

3.1 設定檔定義

config.py

LabelDef (單一標籤)

定義了在視窗上看到的一個具體按鈕或選項（例如:「第一人稱」），裡面記錄了標籤的 ID、顯示名稱 (name)、匯出時實際寫入檔案的代表文字 (tag)、介面顯示的顏色 (color)，以及使用者自訂的快捷鍵 (shortcut)

GroupDef (群組)

將多個 LabelDef 分門別類打包起來。例如建立一個叫做「視角類別」的群組，裡面裝著「第一人稱」、「非第一人稱」等標籤。
定義了群組的 ID、名稱 (name)、描述 (description)，並持有一個 labels 陣列來裝載屬於它的所有單一標籤。

3.1 新增或修改標籤

不用改程式碼，改 configs/annotation_modes/term_definition.json 的 labels：
如：{ "id": "term", "name": "專業術語", "tag": "Term", "color": "#FFE082", "shortcut": "1" }

改完重新啟動程式後，工具列、popup、選單列、右鍵選單
都會自動出現新的標籤。

欄位	用途
id	存進 Annotation.labels 的值；apply_label 收到的 label_id
name	按鈕與選單上顯示的文字
tag	匯出 .txt 時用的標籤字串 (用不到不用管)
color	按鈕與 highlight 的顏色
shortcut	快捷鍵

※ 新增的 label 按下去一樣會呼叫 apply_label：會跳概念對話框、經過重疊檢查（和既有標註不能重疊，見 1-9），最後寫進 labels。
如果加這個 label 只是想要一顆「解釋」之類的按鈕，請在 apply_label 最前面判斷 label_id，改呼叫你自己的函式後直接 return；或改用 2.2 中的一般按鈕。

※ 透過在設定檔中新增另一個 label groups 裡新增標籤雖然可以避開重疊檢查（重疊檢查只看同群組）達到新增一顆「解釋」之類的按鈕的效果(見下一頁)，但需留意：按下去產生的仍然是另一條獨立的標註：不會寫進術語的 explanation_start/end，不符合作業規定，需要自行修改程式邏輯。

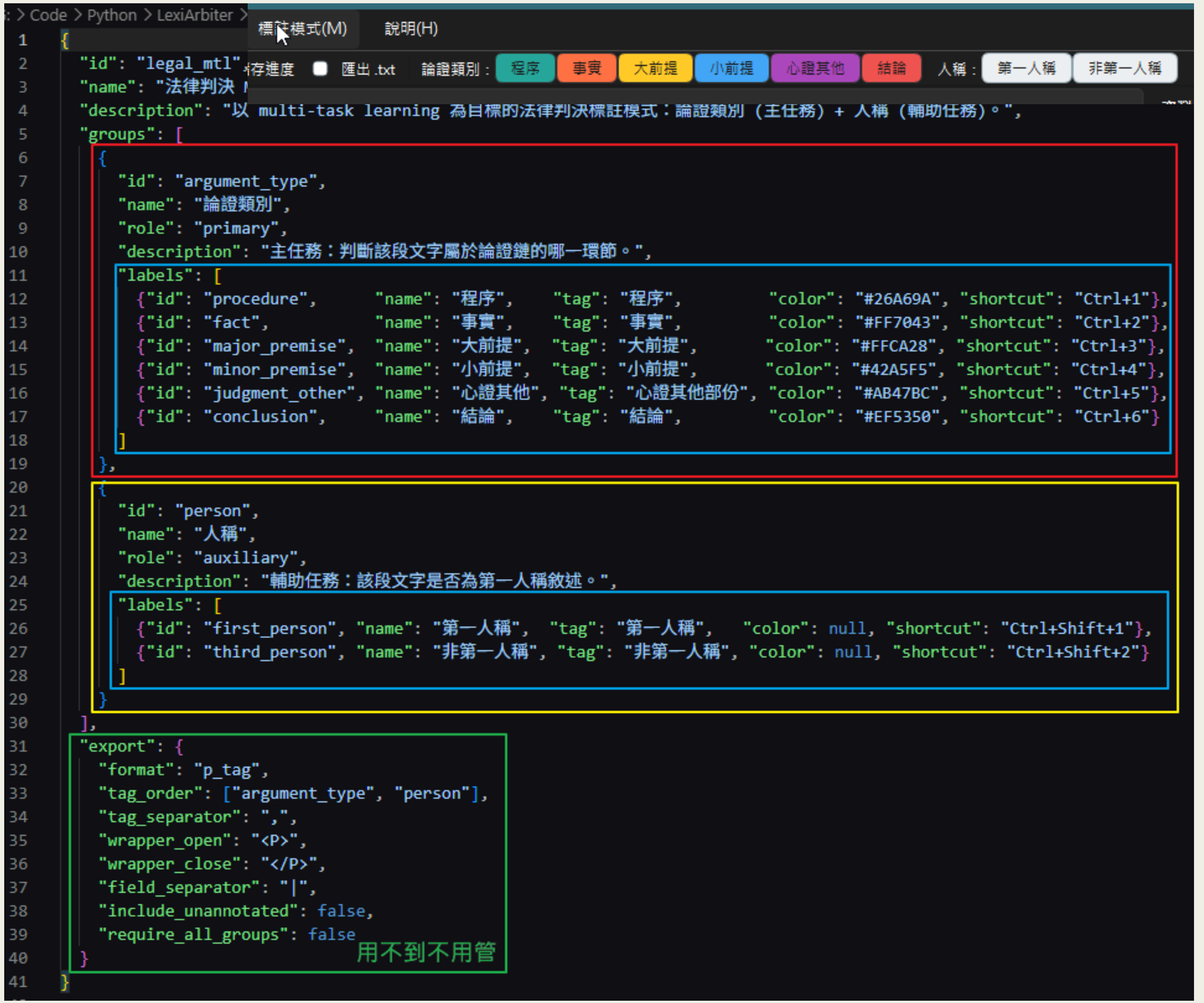
3.1 設定檔定義

右圖是另一個標註任務所用的設定檔，其中紅框、黃框分別代表不同 label groups，裡面各自有自已的標籤 (可對照右上工具列的圖)。

在目前程式架構下，藍框內定義的標籤會進行重疊檢查 (參考1.9)，而紅框、黃框則允許標籤並存。也就是說同一段文字只能被標註為“程序”、“事實”、...等其中之一，而也只能被標註為“第一人稱”或“非第一人稱”其中之一，但可以同時被標註為“程序”+“第一人稱”、“事實”+“第一人稱”等兩個不同 label groups 的排列組合。

如果要透過修改設定檔的方式新增按鈕，要注意你新增的按鈕允不允許重疊，不允許的話要在不同的 label groups 新增。

※ 再次提醒：新增一個 group 雖然可以避開重疊檢查的同時新增一個按鈕，但需自行修改程式邏輯以符合作業規定。



作業目標

- 修改 template 讓使用者可以進行術語-解釋的標註任務，且標註完後依 template 中 model.py 所定義的 Annotation 、Document 格式記錄標註結果。(注意 model.py 中的定義不能更改，且輸出的.lexa檔中欄位必要欄位都必需有值，可參考提供的測試資料比對)。
- 新增功能：在使用者標起來一個術語後，自動標示出文件中所有同樣的術語，注意：
  - 重疊檢查(記得先完善重疊檢查的function)，例如使用者都標出了“損害賠償”、“損害賠償責任”自動標示要如何處理?
  - 自動標示出的術語解釋段落、概念欄位怎麼處理?
- 讓程式儘可能達到 user-friendly。

測試資料

※ 注意：提供的簡易測資目的是為了方便確認程式行為跟預期是否一致、
產出的.lexa是否符合規定，不代表後續實際是這樣標註。

簡易測試資料包含：原始 word 文件(test_example.docx)、標註任務單(對於原始文件做了什麼標註行為)、test_example.lexa (GUI 程式對於標註的行為記錄了什麼)。

測試步驟：

- 用寫好的 GUI 程式讀取 test_example.docx 。
- 依標註任務單上的內容，在寫好的 GUI 程式中做出一模一樣的標註行為。
- 把標註存成.lexa檔
- 跟提供的 test_example.lexa 檔比對 start、end、explanation_start、concept 等欄位是否一致，其中 id 欄位是亂數生成會不一樣是正常的，但 concept_id、broader_concept_id 所指向的概念文字字串應該要是一樣的。

END